

CIRCUITOS DIGITAIS

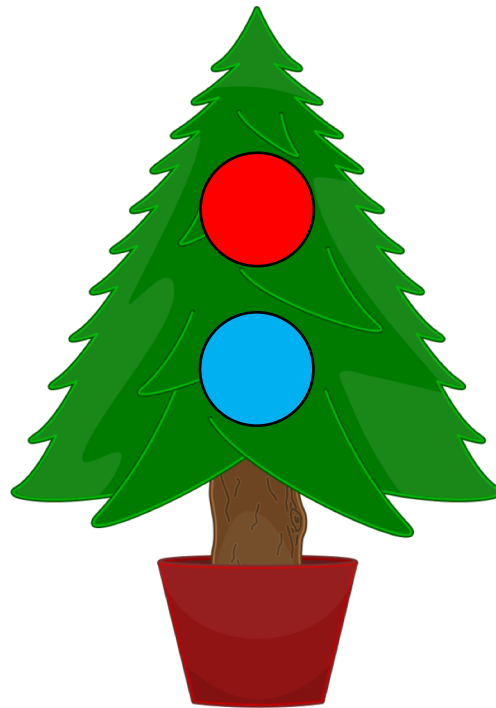
MÁQUINAS DE ESTADO

Prof. Marcelo Grandi Mandelli

`mgmandelli@unb.br`

Exemplo 1

- Projete uma máquina de estados de **Moore** para um sistema de acendimento de luzes de uma árvore de natal:
 - Há duas luzes na árvore de natal: uma **VERMELHA** e uma **AZUL**.



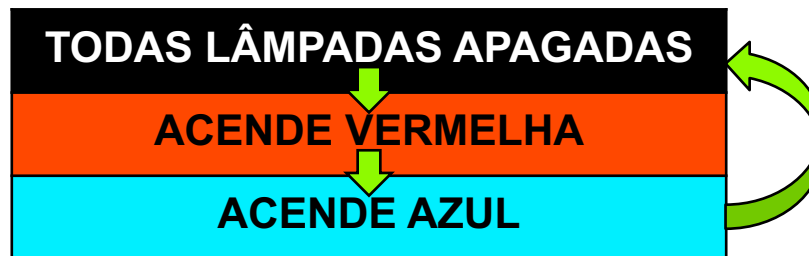
Exemplo 1

- Ao se ligar o sistema todas as luzes iniciam apagadas.
- Há dois modos de acendimento das luzes, controlados por um botão **M** :

- Se **M = 0**, as lâmpadas piscam ao mesmo tempo, seguindo o ciclo:



- Se **M = 1**, somente uma lâmpada é acesa por vez, seguindo o ciclo:



Projeto de Máquinas de Estados Finitos

Passos para o Projeto de Máquinas de Estados:

1. Obter a tabela de transição de estados (pode-se obter através de um diagrama de estados)
2. Codificar os estados → modificar tabela de transição de estados
3. Escolher um flip-flop para o circuito
4. Obter a(s) equação(s) de entrada para o FF escolhido e a(s) equações da(s) saída(s) → simplificar, se necessário
5. Desenhar circuito lógico com FFs e portas lógicas de acordo com as equações booleanas

Exemplo 1

□ Precisamos definir as entradas e saídas do circuito

□ Entradas:

- **M** → define o modo de acendimento das luzes

□ Saídas:

- Luz **VERMELHA** → **LV**

- Estados da saída:

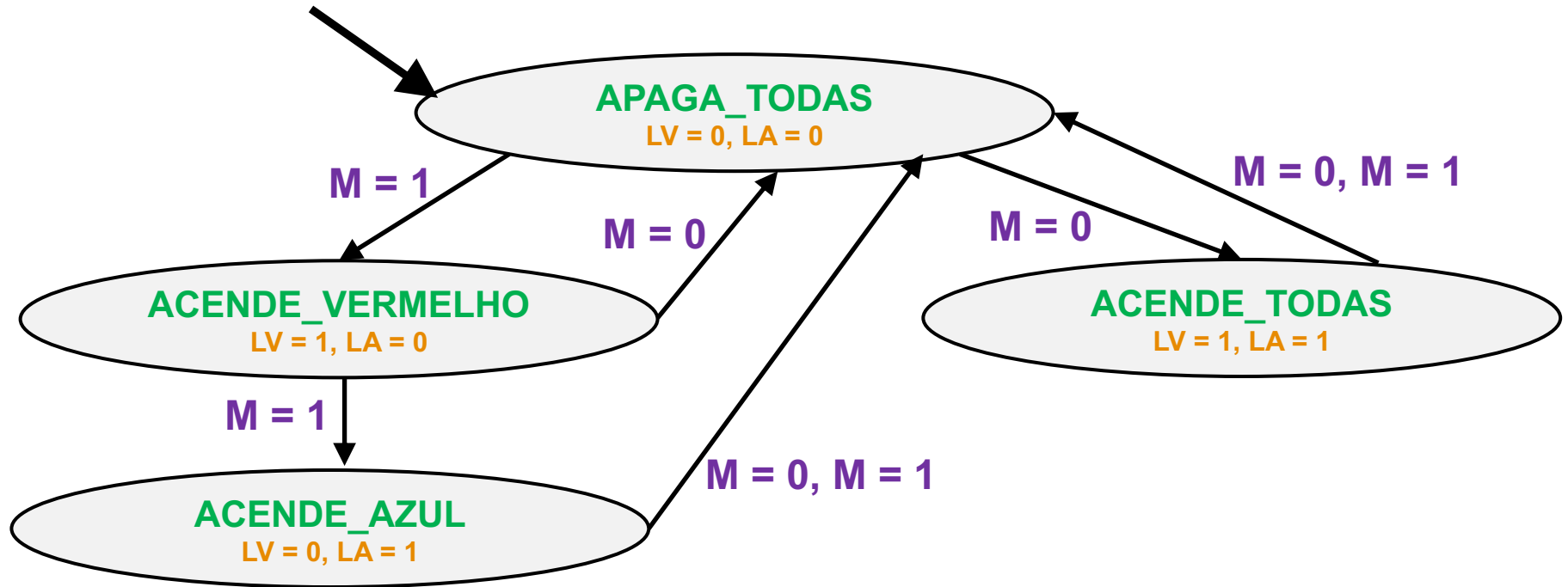
- **LV** = 0 → luz **VERMELHA** apagada
 - **LV** = 1 → luz **VERMELHA** acesa

- Luz **AZUL** → **LA**

- Estados da saída:

- **LA** = 0 → luz **AZUL** apagada
 - **LA** = 1 → luz **AZUL** acesa

Diagrama de Estados



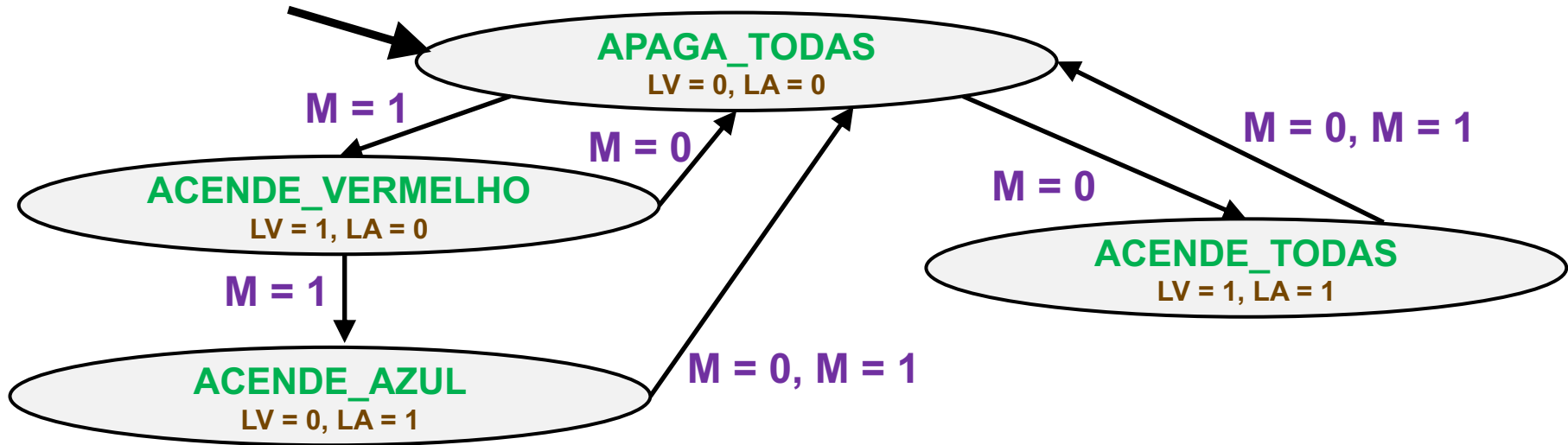
- Se $M = 0$, as lâmpadas piscam ao mesmo tempo, seguindo o ciclo:



- Se $M = 1$, somente uma lâmpada é acesa por vez, seguindo o ciclo:

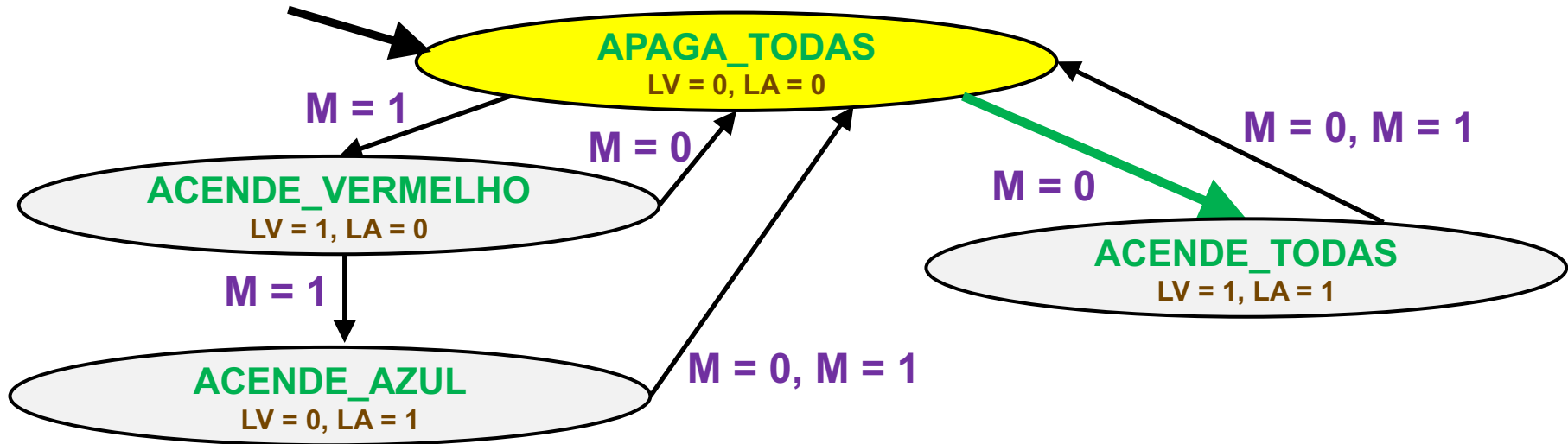


Passo 1 - Obter a tabela de estados



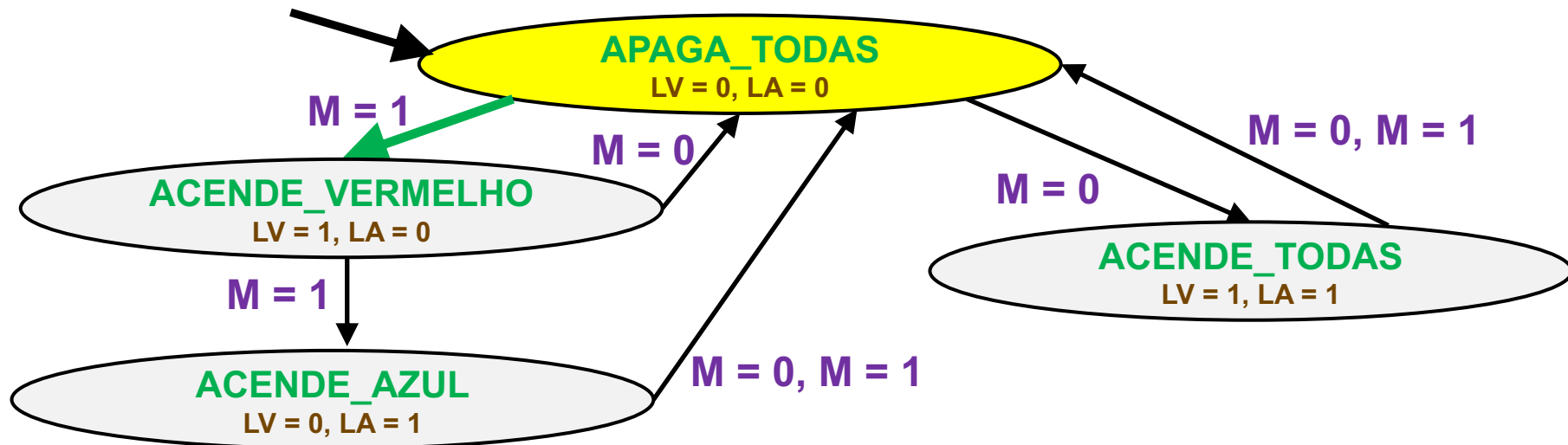
Estado Atual	Entrada	Próximo Estado	Saída	
	M		LV	LA

Passo 1 - Obter a tabela de estados



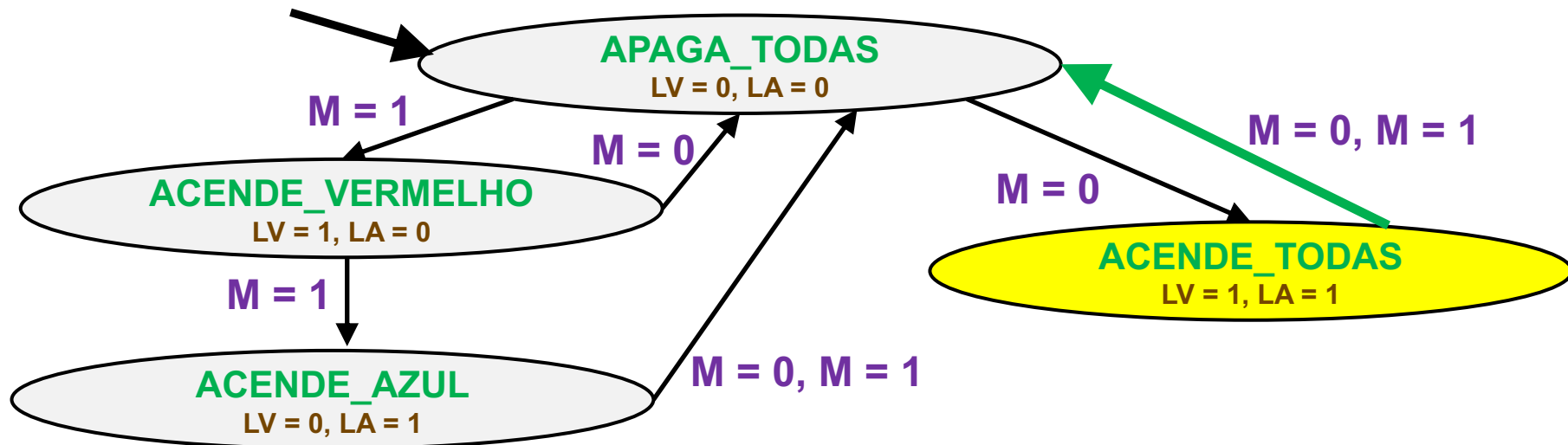
Estado Atual	Entrada	Próximo Estado	Saída	
	M		LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0

Passo 1 - Obter a tabela de estados



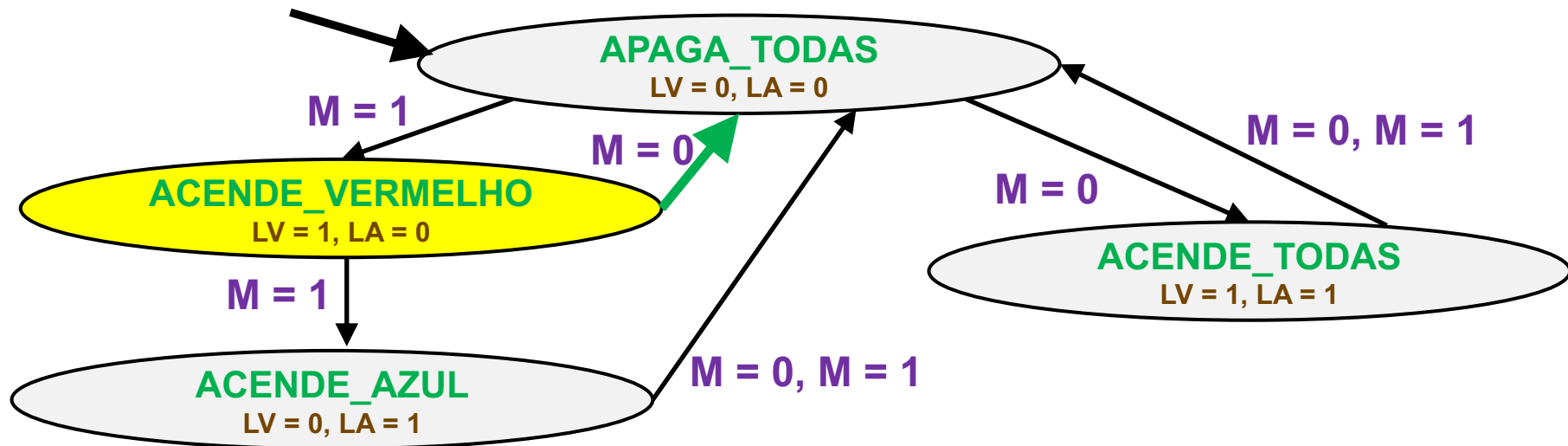
Estado Atual	Entrada	Próximo Estado	Saída	
	M		LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0

Passo 1 - Obter a tabela de estados



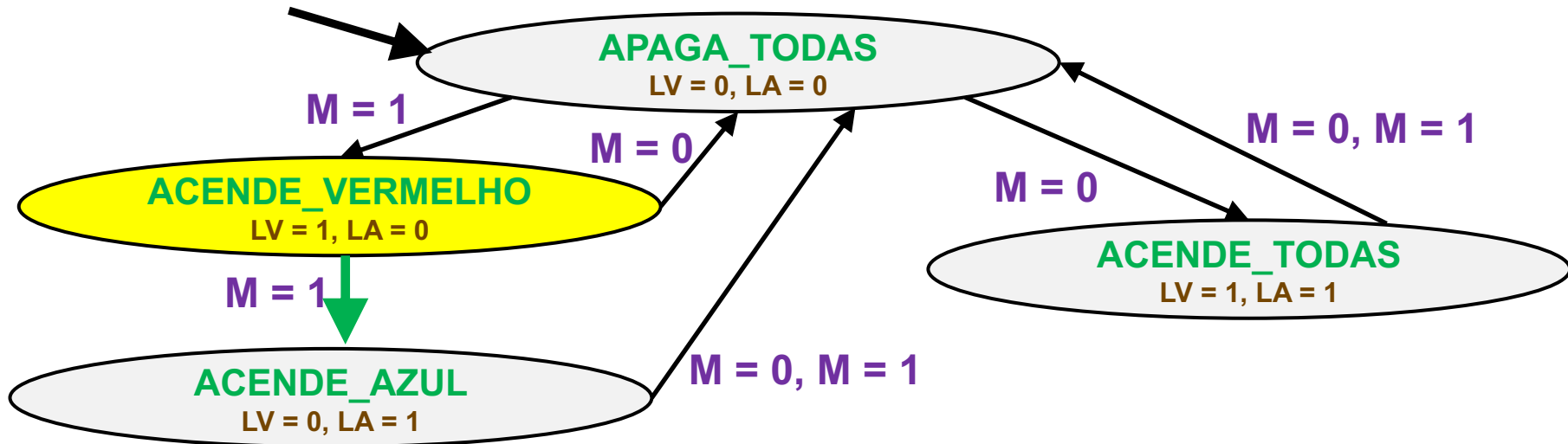
Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0
ACENDE_TODAS	0	APAGA_TODAS	1	1
ACENDE_TODAS	1	APAGA_TODAS	1	1

Passo 1 - Obter a tabela de estados



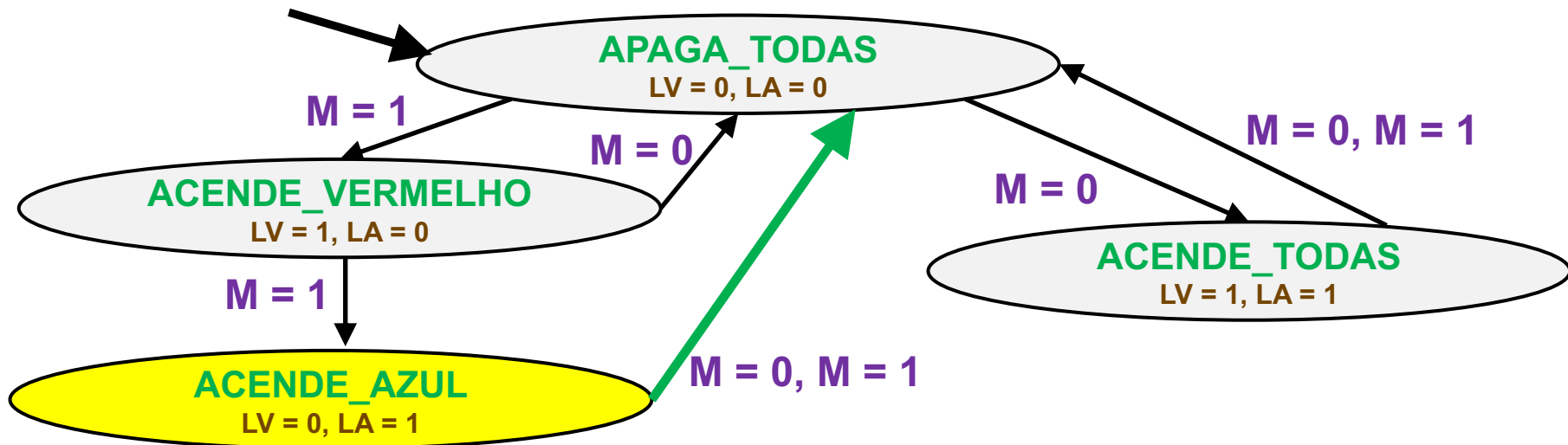
Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0
ACENDE_TODAS	0	APAGA_TODAS	1	1
ACENDE_TODAS	1	APAGA_TODAS	1	1
ACENDE_VERMELHO	0	APAGA_TODAS	1	0

Passo 1 - Obter a tabela de estados



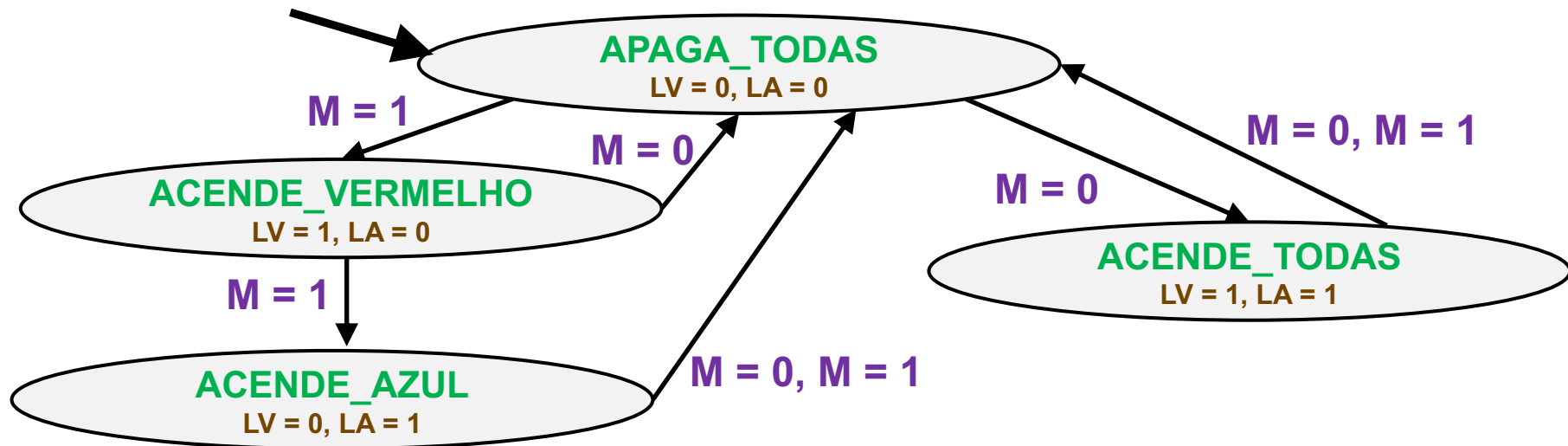
Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0
ACENDE_TODAS	0	APAGA_TODAS	1	1
ACENDE_TODAS	1	APAGA_TODAS	1	1
ACENDE_VERMELHO	0	APAGA_TODAS	1	0
ACENDE_VERMELHO	1	ACENDE_AZUL	1	0

Passo 1 - Obter a tabela de estados



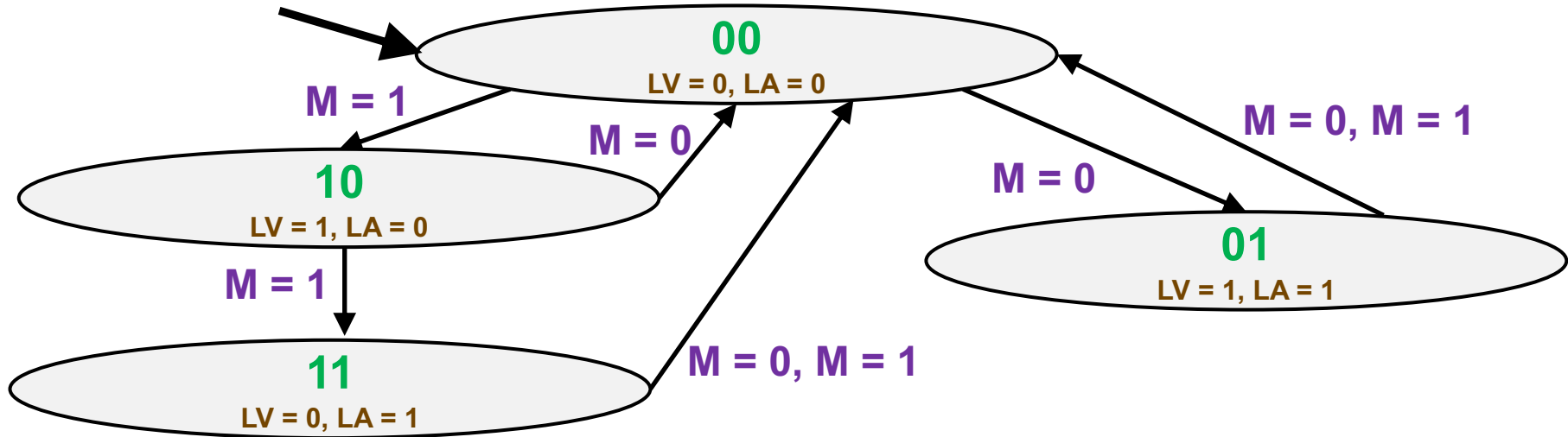
Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0
ACENDE_TODAS	0	APAGA_TODAS	1	1
ACENDE_TODAS	1	APAGA_TODAS	1	1
ACENDE_VERMELHO	0	APAGA_TODAS	1	0
ACENDE_VERMELHO	1	ACENDE_AZUL	1	0
ACENDE_AZUL	0	APAGA_TODAS	0	1
ACENDE_AZUL	1	APAGA_TODAS	0	1

Passo 1 - Obter a tabela de estados



Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0
ACENDE_TODAS	0	APAGA_TODAS	1	1
ACENDE_TODAS	1	APAGA_TODAS	1	1
ACENDE_VERMELHO	0	APAGA_TODAS	1	0
ACENDE_VERMELHO	1	ACENDE_AZUL	1	0
ACENDE_AZUL	0	APAGA_TODAS	0	1
ACENDE_AZUL	1	APAGA_TODAS	0	1

Passo 2 – Codificação dos Estados



Estado Atual		Entrada	Próximo Estado		Saída	
Q_1	Q_0	M	Q_1	Q_0	LV	LA
0	0	0	0	1	0	0
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	1	0	0	1	1
1	0	0	0	0	1	0
1	0	1	1	1	1	0
1	1	0	0	0	0	1
1	1	1	0	0	0	1

CÓDIGO DE CONTAGEM BINÁRIA

Estado	Estado do FF (Q_1Q_0)
APAGA_TODAS	00
ACENDE_TODAS	01
ACENDE_VERMELHO	10
ACENDE_AZUL	11

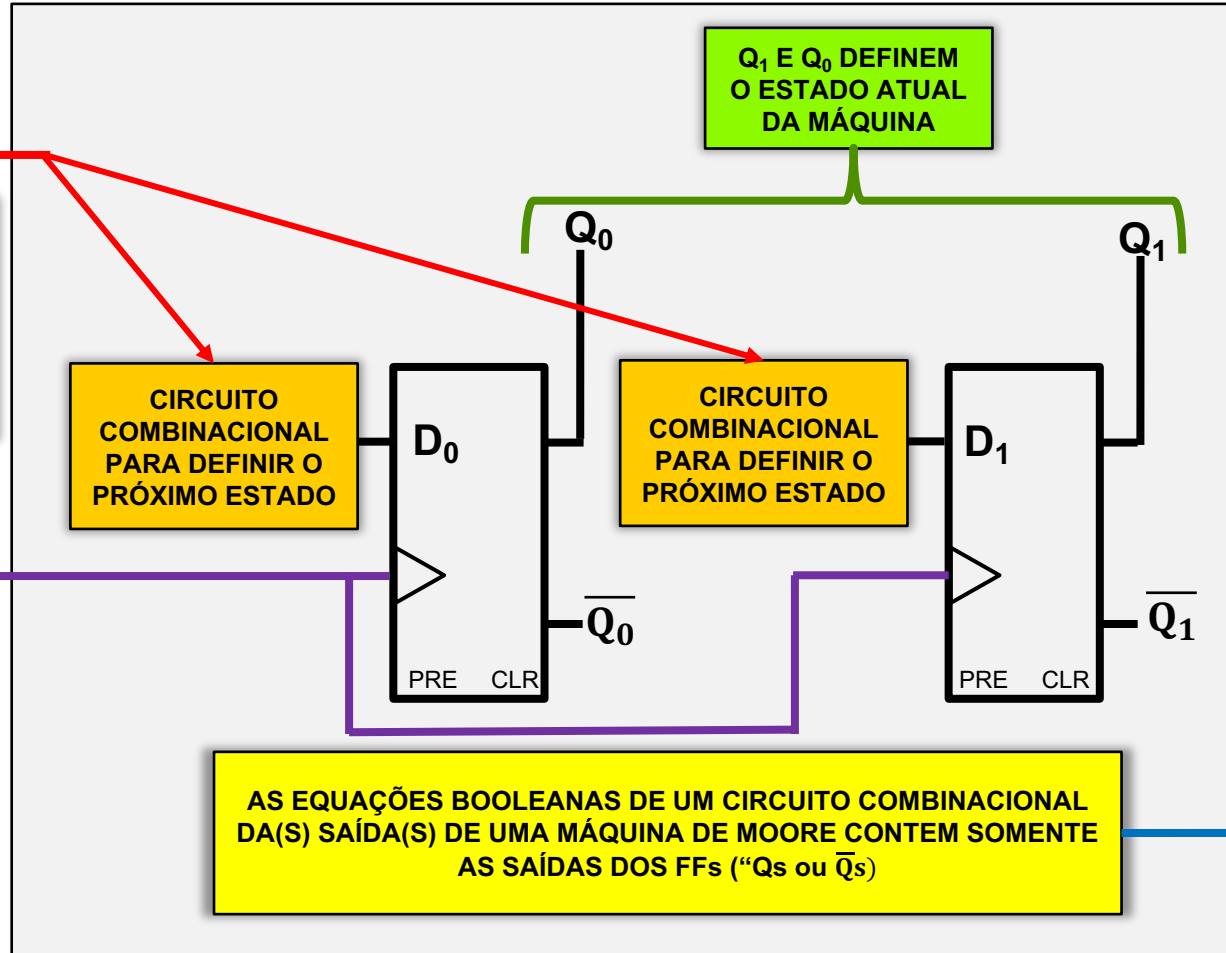
Passo 3 – Escolha do Flip-Flop

□ NESTE EXEMPLO VAMOS UTILIZAR O FLIP-FLOP D

ENTRADA(S)

EM UMA MÁQUINA DE MOORE, AS ENTRADAS SÃO USADAS SOMENTE NOS CIRCUITOS COMBINACIONAIS DE PRÓXIMO ESTADO

CLOCK



Passo 4 – Equações de Entrada dos FFs

Estado Atual		Entrada	Próximo Estado		Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	Q_1	Q_0	LV	LA	D_1	D_0
0	0	0	0	1	0	0		
0	0	1	1	0	0	0		
0	1	0	0	0	1	1		
0	1	1	0	0	1	1		
1	0	0	0	0	1	0		
1	0	1	1	1	1	0		
1	1	0	0	0	0	1		
1	1	1	0	0	0	1		

Passo 4 – Equações de Entrada dos FFs

Estado Atual		Entrada	Próximo Estado		Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	Q_1	Q_0	LV	LA	D_1	D_0
0	0	0	0	1	0	0		
0	0	1	1	0	0	0		
0	1	0	0	0	1	1		
0	1	1	0	0	1	1		
1	0	0	0	0	1	0		
1	0	1	1	1	1	0		
1	1	0	0	0	0	1		
1	1	1	0	0	0	1		

O valor de Q_1 deve se manter em 0 no próximo estado.

Qual o valor de D para que isso aconteça?

Passo 4 – Equações de Entrada dos FFs

Estado Atual		Entrada	Próximo Estado		Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	Q_1	Q_0	LV	LA	D_1	D_0
0	0	0	0	1	0	0	0	
0	0	1	1	0	0	0		
0	1	0	0	0	1	1		
0	1	1	0	0	1	1		
1	0	0	0	0	1	0		
1	0	1	1	1	1	0		
1	1	0	0	0	0	1		
1	1	1	0	0	0	1		

CLK	D	Q
$\neq \uparrow$	X	Q
$\uparrow$	0	0
$\uparrow$	1	1

Passo 4 – Equações de Entrada dos FFs

Estado Atual		Entrada	Próximo Estado		Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	Q_1	Q_0	LV	LA	D_1	D_0
0	0	0	0	1	0	0	0	
0	0	1	1	0	0	0		
0	1	0	0	0	1	1		
0	1	1	0	0	1	1		
1	0	0	0	0	1	0		
1	0	1	1	1	1	0		
1	1	0	0	0	0	1		
1	1	1	0	0	0	1		

O valor de Q_0 deve mudar de 0 para 1 no próx. estado.

Qual o valor de D para que isso aconteça?

Passo 4 – Equações de Entrada dos FFs

Estado Atual		Entrada	Próximo Estado		Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	Q_1	Q_0	LV	LA	D_1	D_0
0	0	0	0	1	0	0	0	1
0	0	1	1	0	0	0		
0	1	0	0	0	1	1		
0	1	1	0	0	1	1		
1	0	0	0	0	1	0		
1	0	1	1	1	1	0		
1	1	0	0	0	0	1		
1	1	1	0	0	0	1		

CLK	D	Q
$\neq \uparrow$	X	Q
$\uparrow$	0	0
$\uparrow$	1	1

Passo 4 – Equações de Entrada dos FFs

Estado Atual		Entrada	Próximo Estado		Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	Q_1	Q_0	LV	LA	D_1	D_0
0	0	0	0	1	0	0	0	1
0	0	1	1	0	0	0	1	0
0	1	0	0	0	1	1	0	0
0	1	1	0	0	1	1	0	0
1	0	0	0	0	1	0	0	0
1	0	1	1	1	1	0	1	1
1	1	0	0	0	0	1	0	0
1	1	1	0	0	0	1	0	0



Para o FF tipo D, a tabela das equações de entrada é igual a tabela do próximo estado!

Passo 4 – Equações de Entrada dos FFs

Estado Atual		Entrada	Próximo Estado	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M		LV	LA	D_1	D_0
0	0	0		0	0	0	1
0	0	1	1	0	0	1	0
0	1	0	0	1	1	0	0
0	1	1	0	1	1	0	0
1	0	0		1	0	0	0
1	0	1		1	0	1	1
1	1	0		0	1	0	0
1	1	1		0	1	0	0

DEPOIS DE PREENCHER A PARTE DA TABELA DE EQUAÇÕES DE ENTRADA, VOCÊ PODE DESCARTAR A PARTE DE PRÓXIMO ESTADO

Passo 4 – Equações de Entrada dos FFs

Estado Atual		Entrada	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	LV	LA	D_1	D_0
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	0	1	1	0	0
0	1	1	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	0
1	1	1	0	1	0	0

		$Q_0 M$			
		00	01	11	10
Q_1	0	0 0	1 1	0 3	0 2
	1	0 4	1 5	0 7	0 6

$$D_1 = \overline{Q_0} M$$

Passo 4 – Equações de Entrada dos FFs

Estado Atual		Entrada	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	LV	LA	D_1	D_0
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	0	1	1	0	0
0	1	1	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	0
1	1	1	0	1	0	0

		$Q_0 M$			
Q_1		00	01	11	10
	0	1	0	0	0
	1	0	1	0	0

$$D_0 = \overline{Q_1} \overline{Q_0} \overline{M} + Q_1 \overline{Q_0} M = \overline{Q_0} (Q_1 \text{ XNOR } M)$$

Passo 4 – Equações de Entrada dos FFs

Estado Atual		Entrada	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	LV	LA	D_1	D_0
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	0	1	1	0	0
0	1	1	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	0
1	1	1	0	1	0	0

Equações de Entrada para o FF tipo D :

$$D_1 = \overline{Q_0}M$$

$$D_0 = \overline{Q_0}(Q_1 \text{ XNOR } M)$$

Passo 4 – Equações da(s) Saída(s)

Estado Atual		Entrada	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	LV	LA	D_1	D_0
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	0	1	1	0	0
0	1	1	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	0
1	1	1	0	1	0	0

$$LV = Q_1 \overline{Q_0} + \overline{Q_1} Q_0 = Q_1 \text{ XOR } Q_0$$

MÁQUINA DE MOORE → SAÍDA DEPENDE SÓ DO ESTADO ATUAL

Passo 4 – Equações da(s) Saída(s)

Estado Atual		Entrada	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	LV	LA	D_1	D_0
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	0	1	1	0	0
0	1	1	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	0
1	1	1	0	1	0	0

$$LA = \overline{Q_1}Q_0 + Q_1Q_0 = Q_0$$

MÁQUINA DE MOORE → SAÍDA DEPENDE SÓ DO ESTADO ATUAL

Passo 4 – Equações da(s) Saída(s)

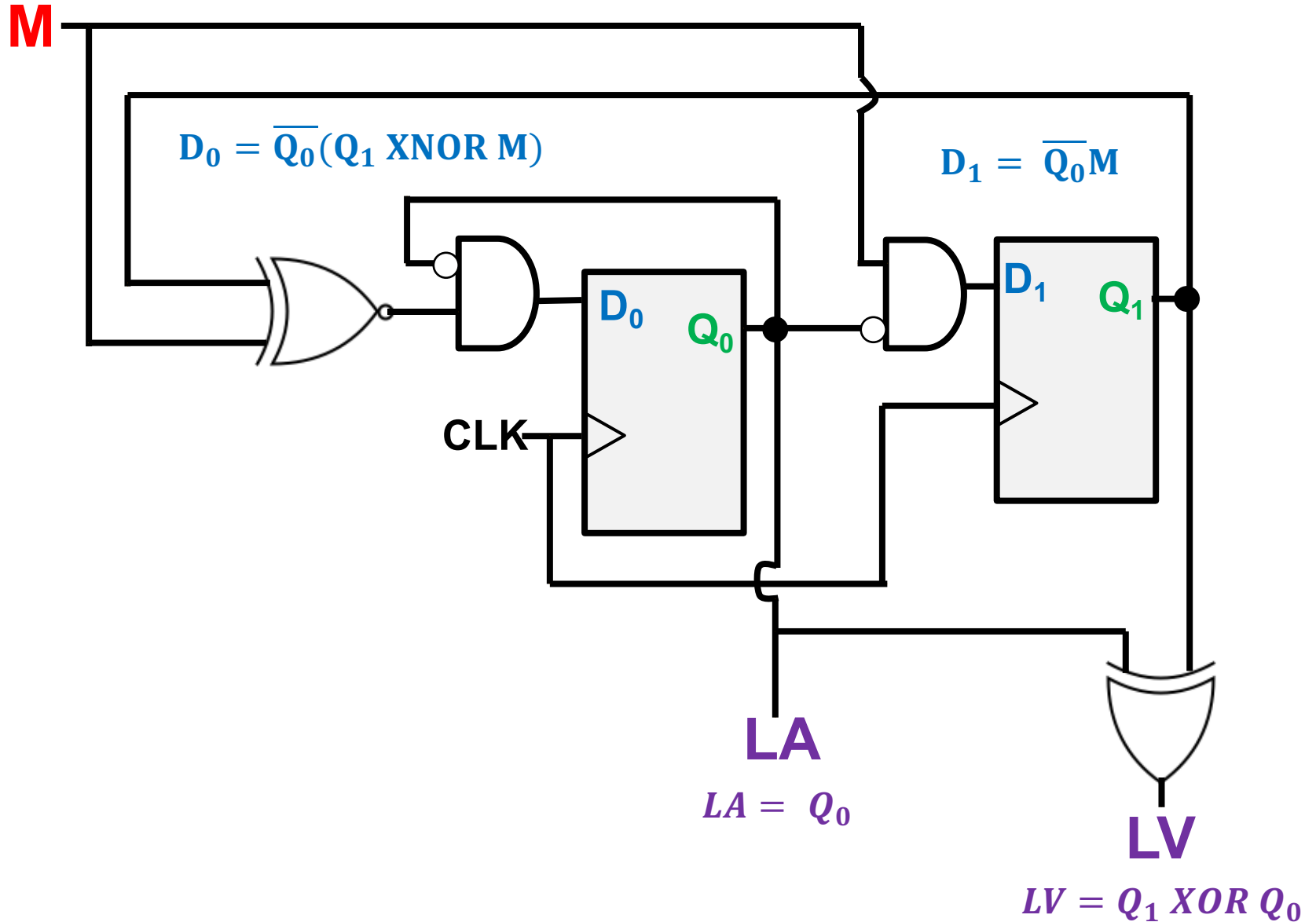
Estado Atual		Entrada	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	LV	LA	D_1	D_0
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	0	1	1	0	0
0	1	1	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	0
1	1	1	0	1	0	0

Equações de Saída :

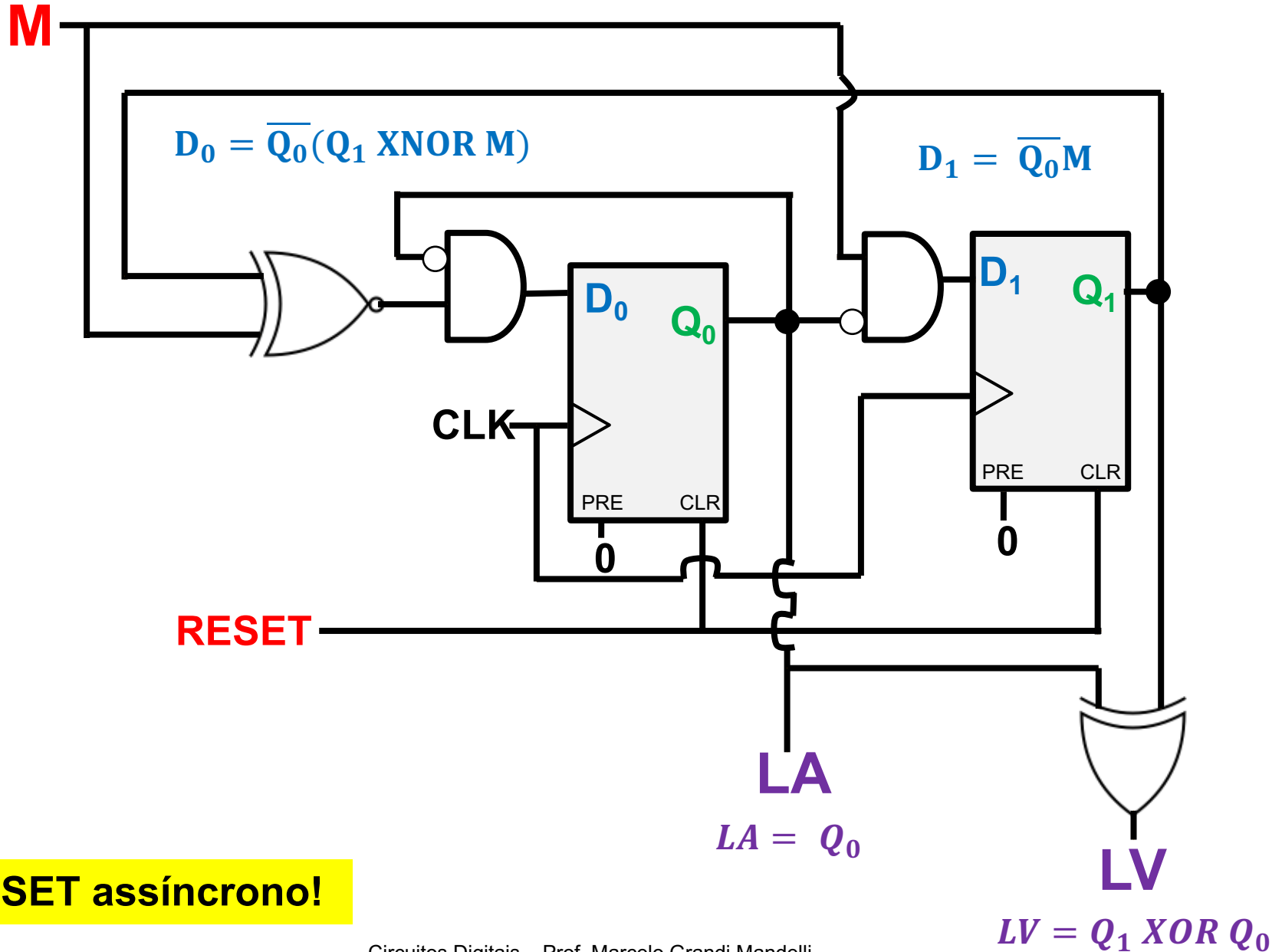
$$LV = Q_1 \text{ XOR } Q_0$$

$$LA = Q_0$$

Passo 5 – Circuito



Passo 5 – Circuito → RESET



Exemplo 2

- Implemente as Equações de Entrada (que definem o próximo estado) e Equações de Saída do exemplo anterior utilizando uma memória ROM

Tabela de Estados do Exemplo 1

Estado Atual		Entrada	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	LV	LA	D_1	D_0
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	0	1	1	0	0
0	1	1	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	0
1	1	1	0	1	0	0

PARA A IMPLEMENTAÇÃO DO CIRCUITO DO EXEMPLO ANTERIOR UTILIZANDO UMA MEMÓRIA ROM, NECESSITAMOS DA TABELA ESTADOS OBTIDA ANTERIORMENTE!

Circuitos combinacionais

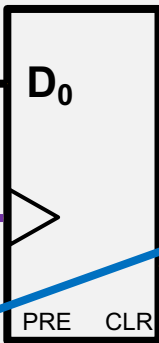
UMA ROM IMPLEMENTARÁ OS CIRCUITOS COMBINACIONAIS

ENTRADA(S)

EM UMA MÁQUINA DE MOORE, AS ENTRADAS SÃO USADAS SOMENTE NOS CIRCUITOS COMBINACIONAIS DE PRÓXIMO ESTADO

Q_1 E Q_0 DEFINEM O ESTADO ATUAL DA MÁQUINA

CIRCUITO COMBINACIONAL PARA DEFINIR O PRÓXIMO ESTADO



CIRCUITO COMBINACIONAL PARA DEFINIR O PRÓXIMO ESTADO



CLOCK

OS CIRCUITO COMBINACIONAIS SERÃO IMPLEMENTADOS UTILIZANDO UMA ROM!

AS EQUAÇÕES BOOLEANAS DE UM CIRCUITO COMBINACIONAL DA(S) SAÍDA(S) DE UMA MÁQUINA DE MOORE CONTEM SOMENTE AS SAÍDAS DOS FFs (" Q_s ou $\bar{Q}_s$ ")

SAÍDA(S)

ROM

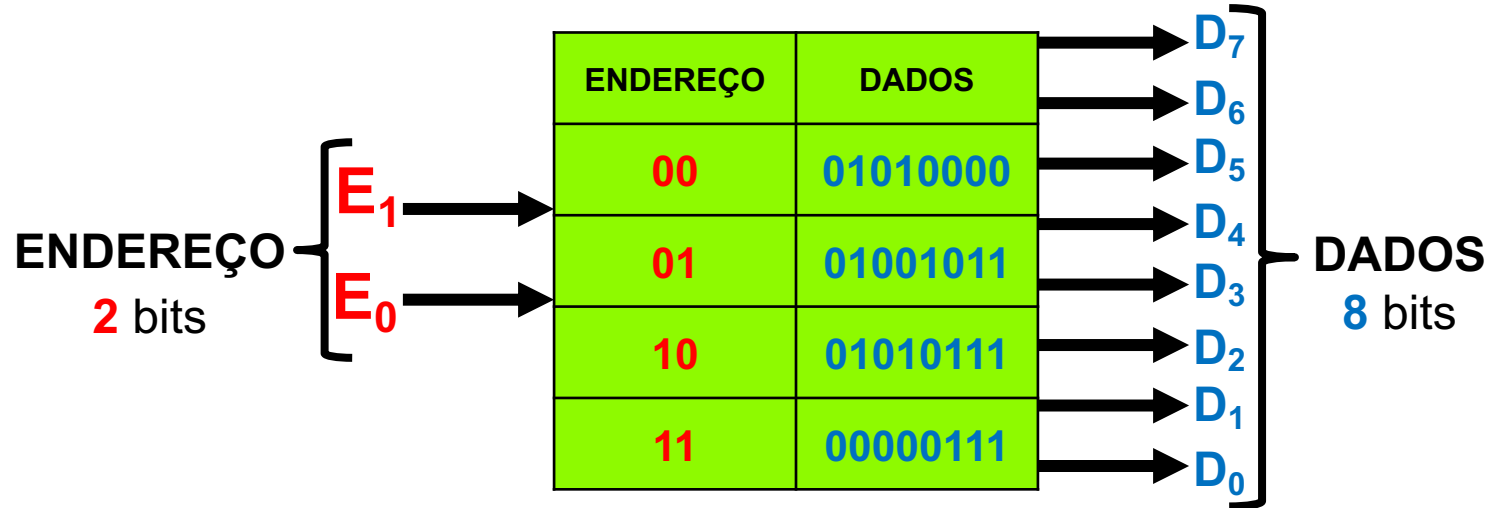
□ Read Only Memory



- Dados tem tamanho palavra **M** bits
- Com **N** bits de endereço é possível armazenar 2^N palavras de dados
- Tamanho da ROM será **M** x 2^N bits

ROM

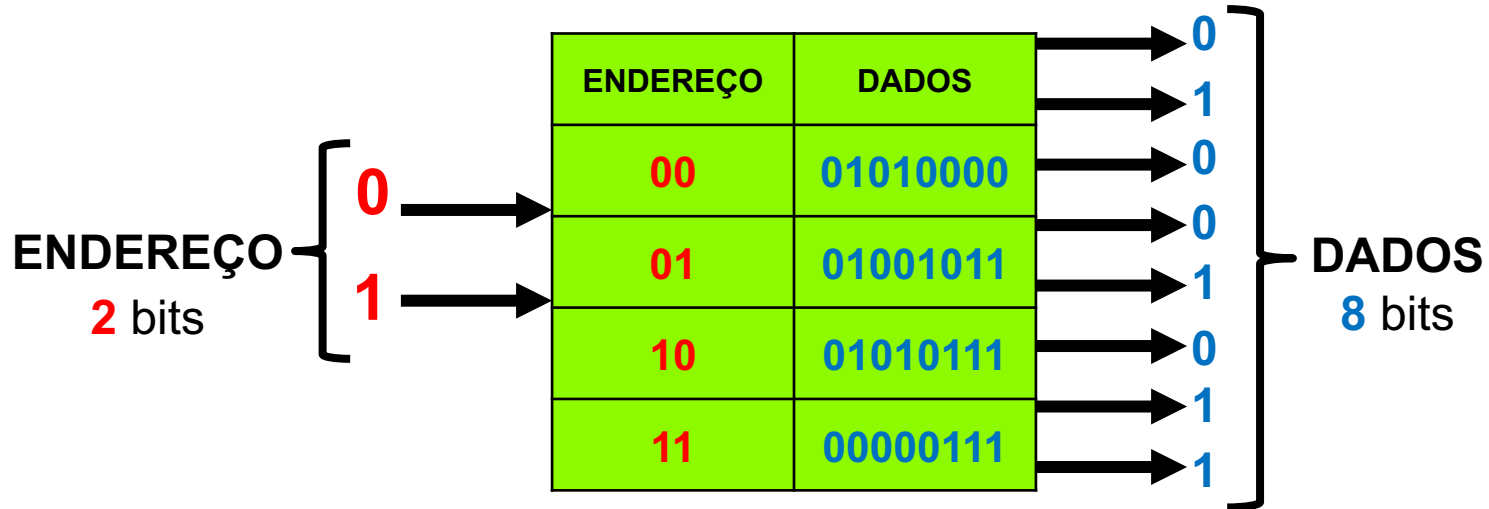
- Exemplo: 2 de endereço e 8 bits de tamanho de dados



- Dados tem tamanho palavra 8 bits
- Com 2 bits de endereço é possível armazenar $2^2 = 4$ palavras de dados
- Tamanho da ROM será $8 \times 2^2 = 8 \times 4 = 32$ bits

ROM

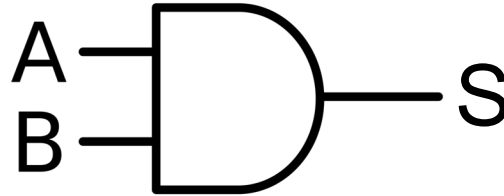
- Exemplo: 2 de endereço e 8 bits de tamanho de dados



- Dados tem tamanho palavra 8 bits
- Com 2 bits de endereço é possível armazenar $2^2 = 4$ palavras de dados
- Tamanho da ROM será $8 \times 2^2 = 8 \times 4 = 32$ bits

□ Implementação de Circuitos Combinacionais

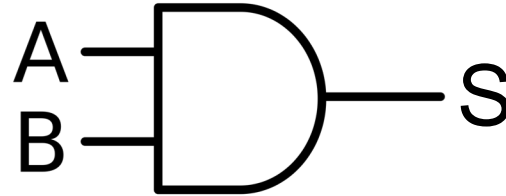
- Exemplo: Porta AND



Técnica chamada Look Up Table (LUT)

□ Implementação de Circuitos Combinacionais

- Exemplo: Porta AND



Técnica chamada Look Up Table (LUT)

Voltando a nosso exemplo

Estado Atual		Entrada	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	LV	LA	D_1	D_0
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	0	1	1	0	0
0	1	1	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	0
1	1	1	0	1	0	0

Tabela de Estados com ROM

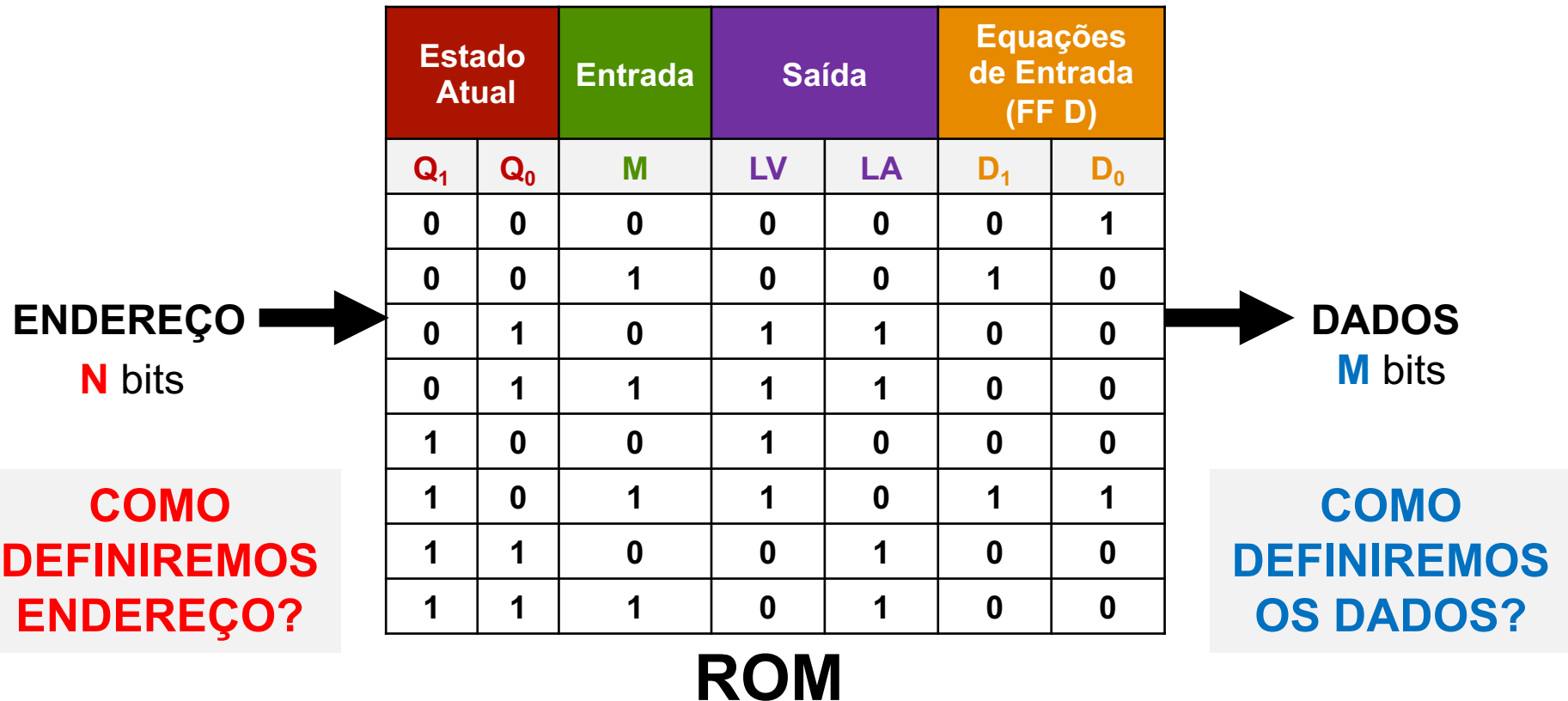


Tabela de Estados com ROM

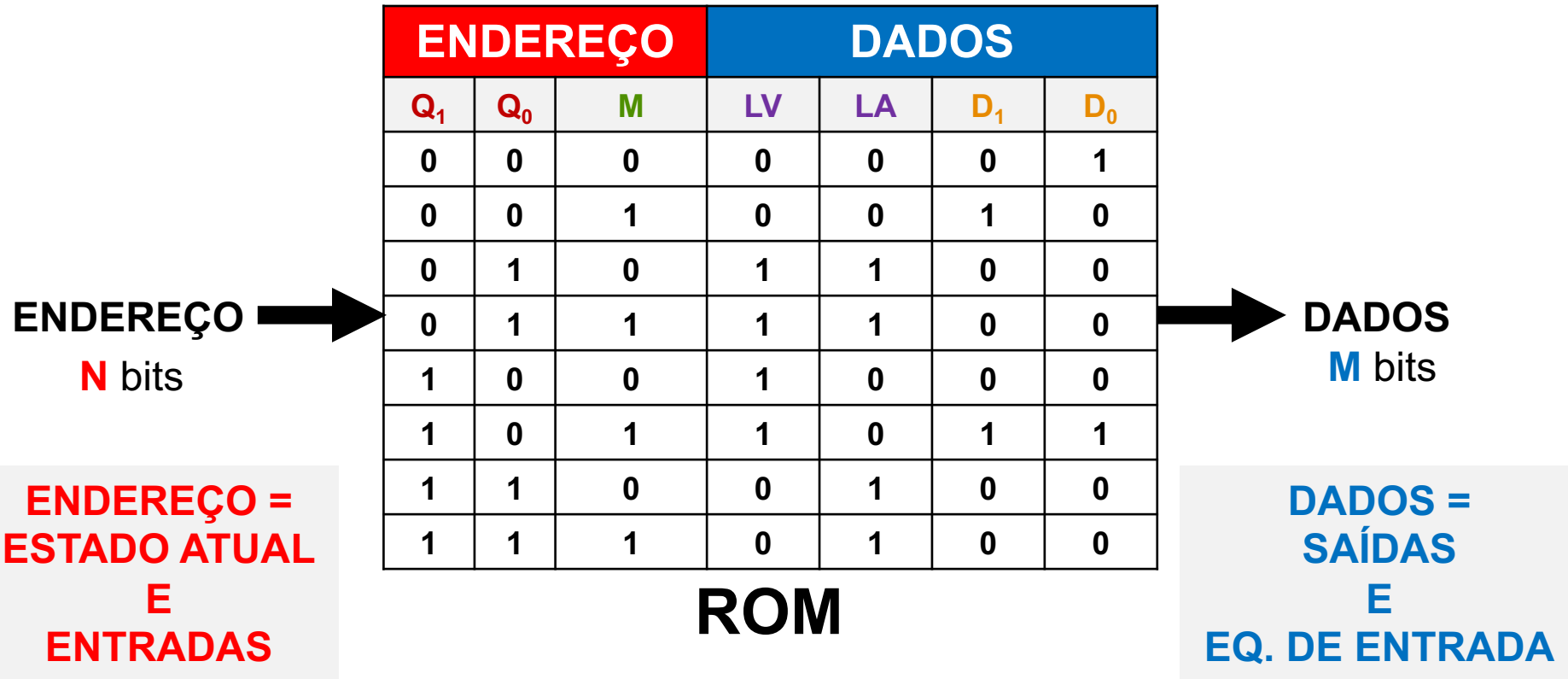
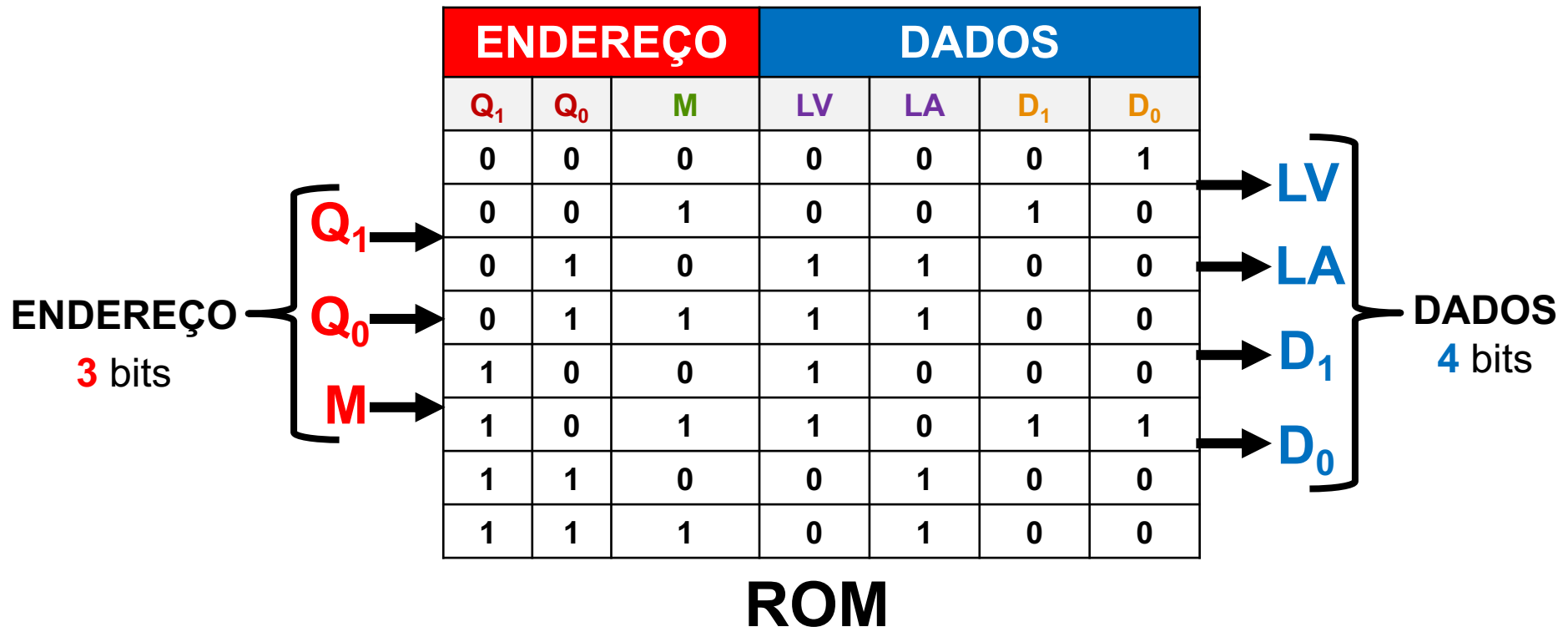
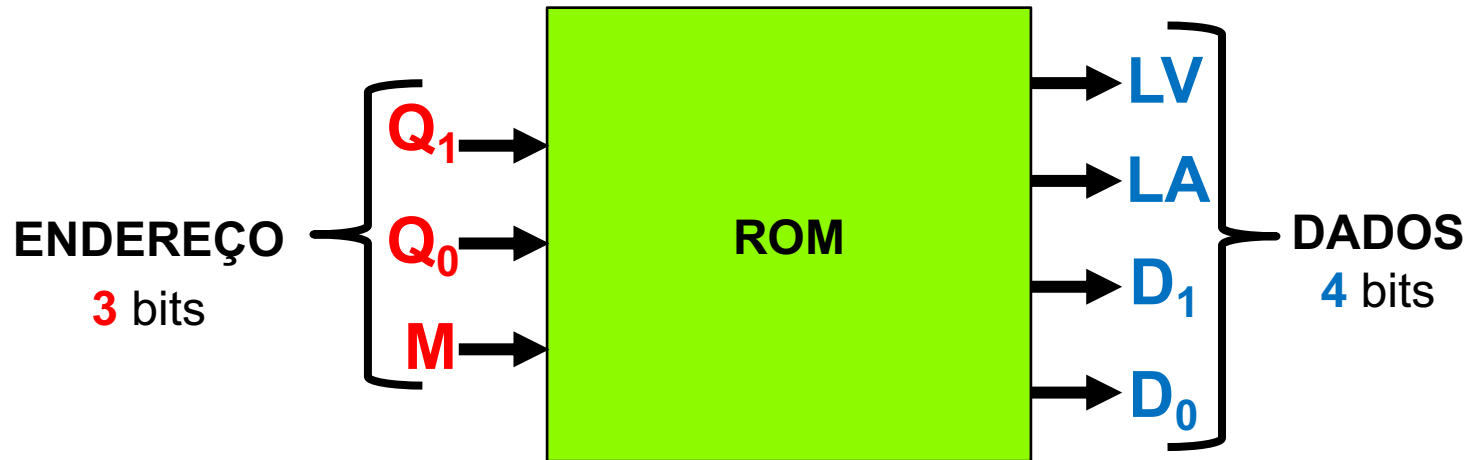


Tabela de Estados com ROM

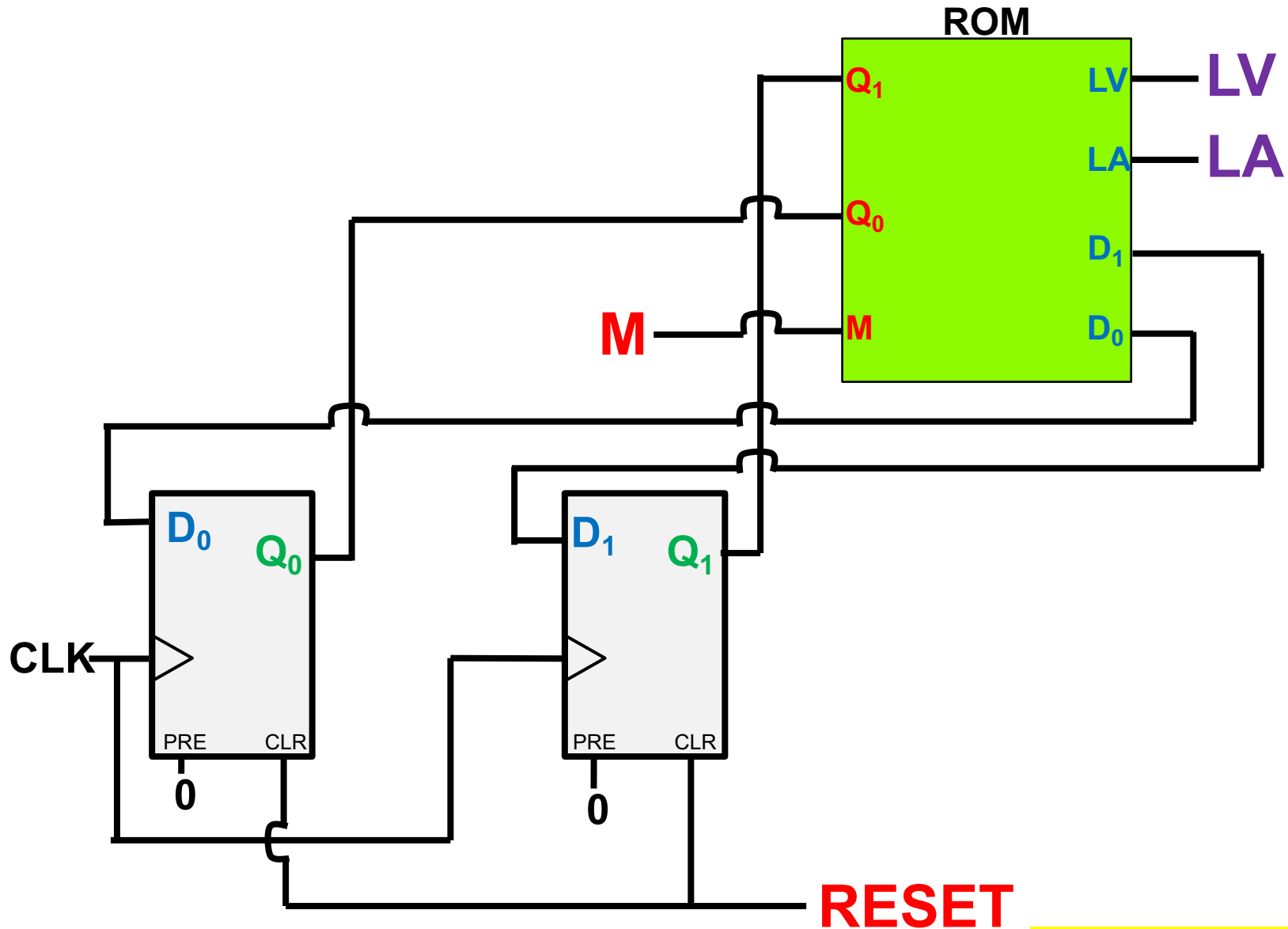


Tamanho da ROM será $4 \times 2^3 = 4 \times 8 = 32$ bits

Tabela de Estados com ROM



Circuito



RESET assíncrono!

ROM do Exemplo 2 em SystemVerilog

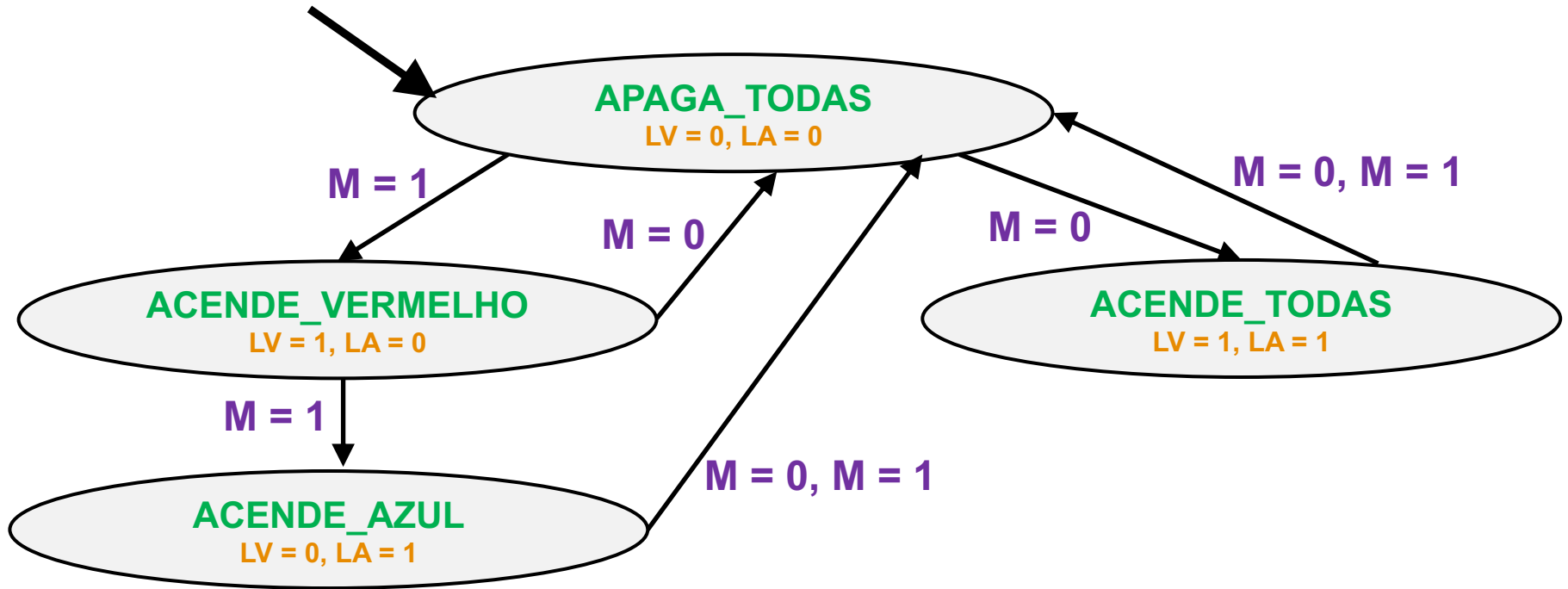
ENDEREÇO			DADOS			
Q ₁	Q ₀	M	LV	LA	D ₁	D ₀
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	0	1	1	0	0
0	1	1	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	0
1	1	1	0	1	0	0

```
module rom(  
    input [2:0] endereco,  
    output [3:0] dados);  
  
    always_comb  
        casex (endereco)  
            3'b 000: dados = 4'b 0001;  
            3'b 001: dados = 4'b 0010;  
            3'b 010: dados = 4'b 1100;  
            3'b 011: dados = 4'b 1100;  
            3'b 100: dados = 4'b 1000;  
            3'b 101: dados = 4'b 1011;  
            3'b 110: dados = 4'b 0100;  
            3'b 111: dados = 4'b 0100;  
            default: dados = 4'b 0000;  
        endcase  
  
endmodule
```

Exemplo 3

- Implemente as Equações de Entrada (que definem o próximo estado) e das Equações de Saída do Exemplo 1 utilizando uma memória ROM
- **Utilize "don't cares"(X) no Diagrama de Estados, quando possível**

Diagrama de Estados do Exemplo 1



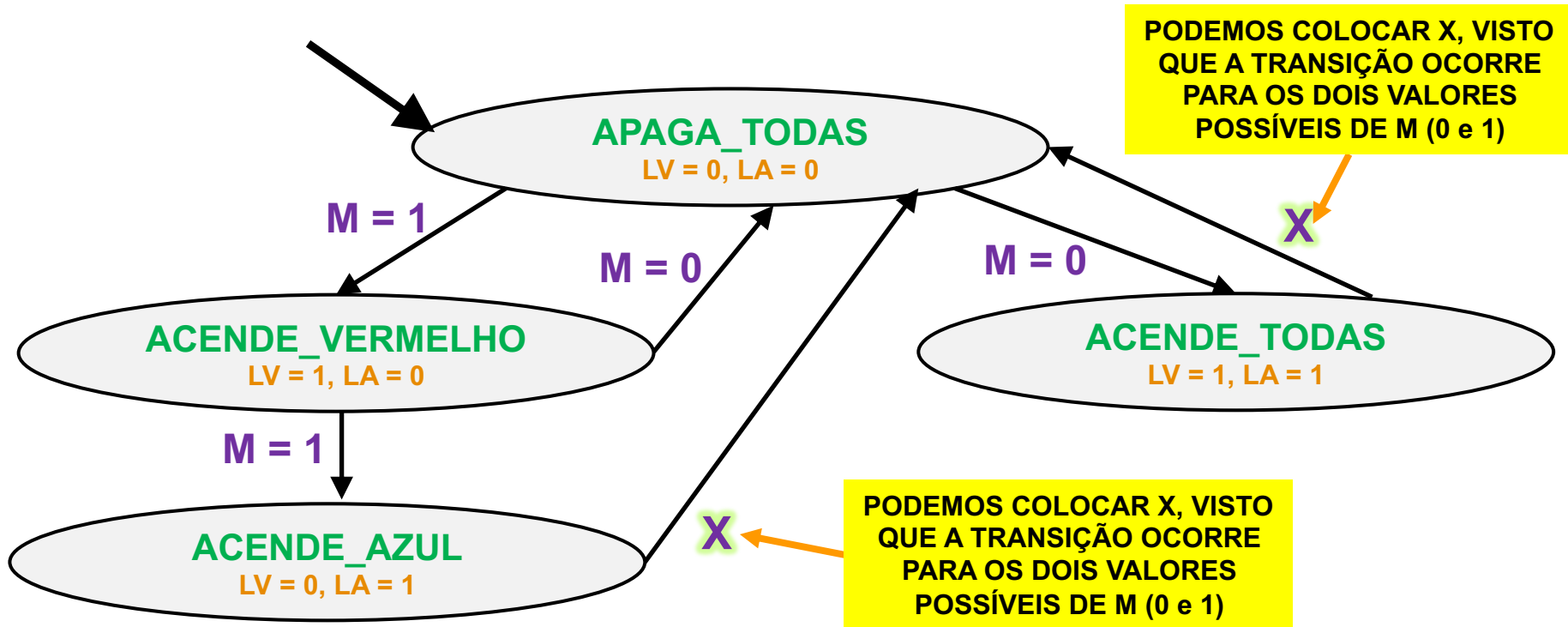
- Se $M = 0$, as lâmpadas piscam ao mesmo tempo, seguindo o ciclo:



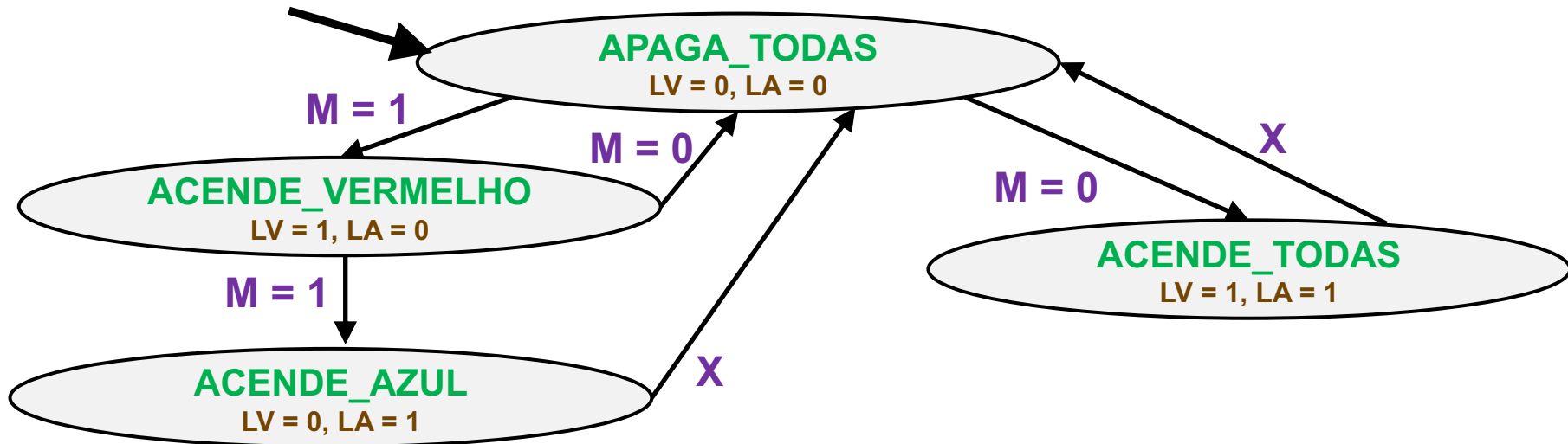
- Se $M = 1$, somente uma lâmpada é acesa por vez, seguindo o ciclo:



Diagrama de Estados do Exemplo 1

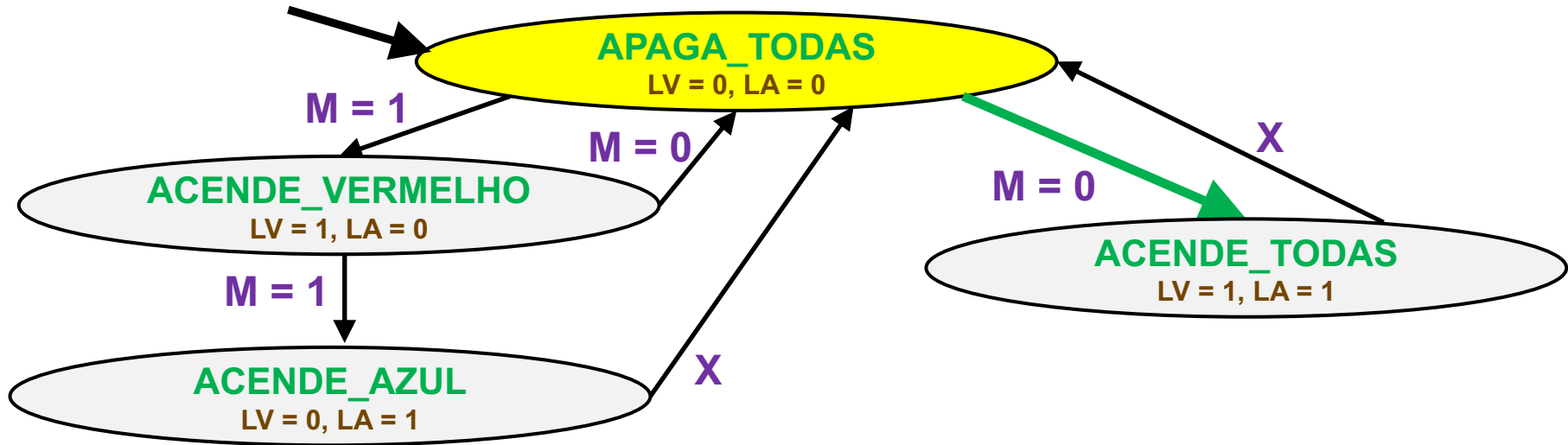


Obtenção da tabela de estados



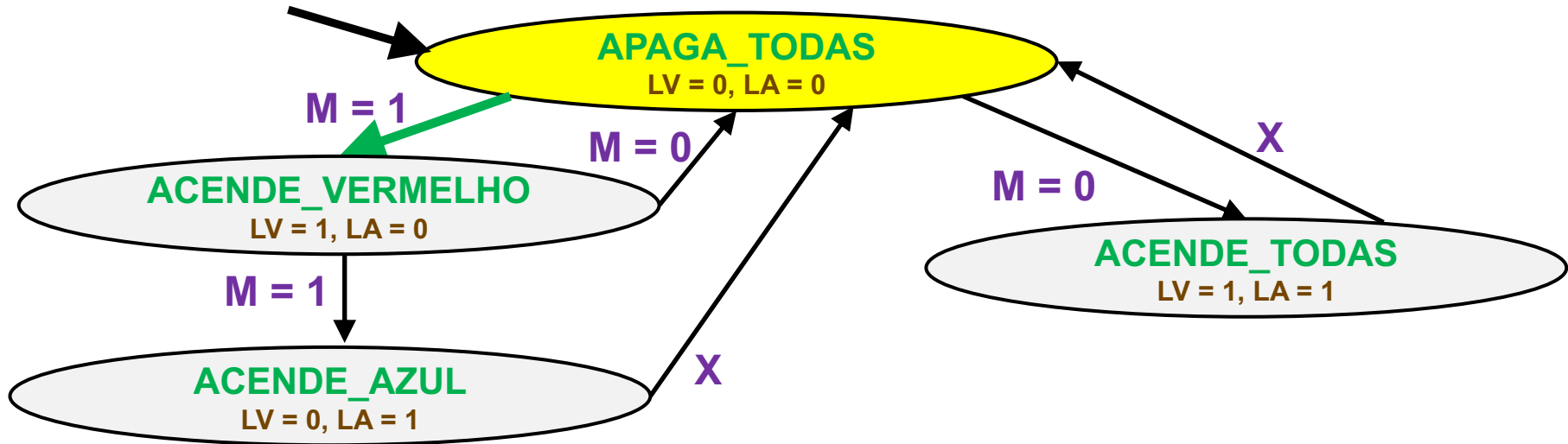
Estado Atual	Entrada	Próximo Estado	Saída	
	M		LV	LA

Obtenção da tabela de estados



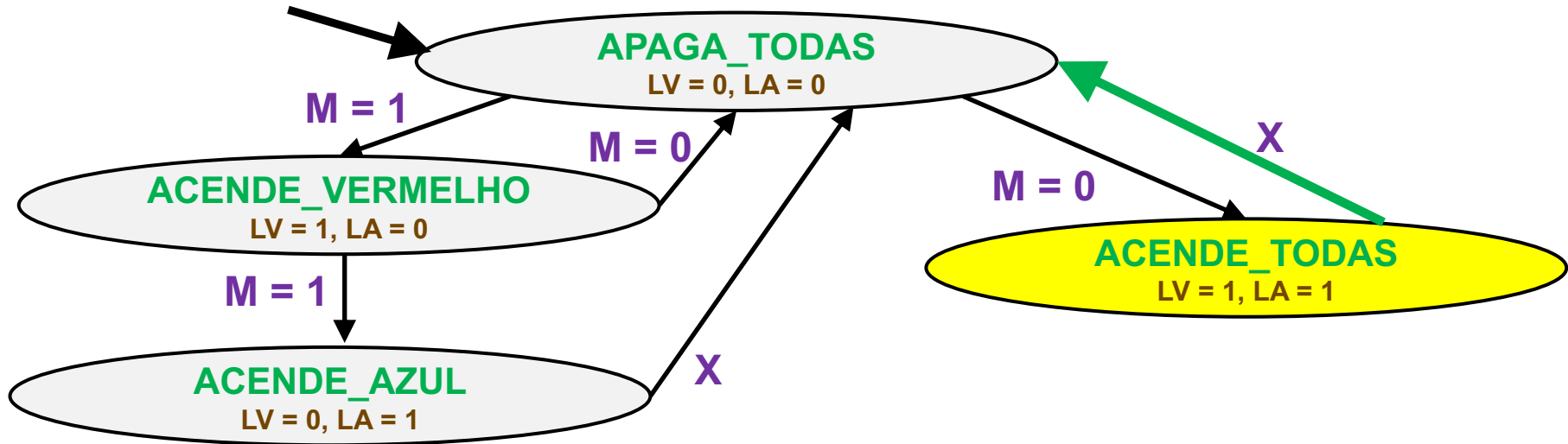
Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0

Obtenção da tabela de estados



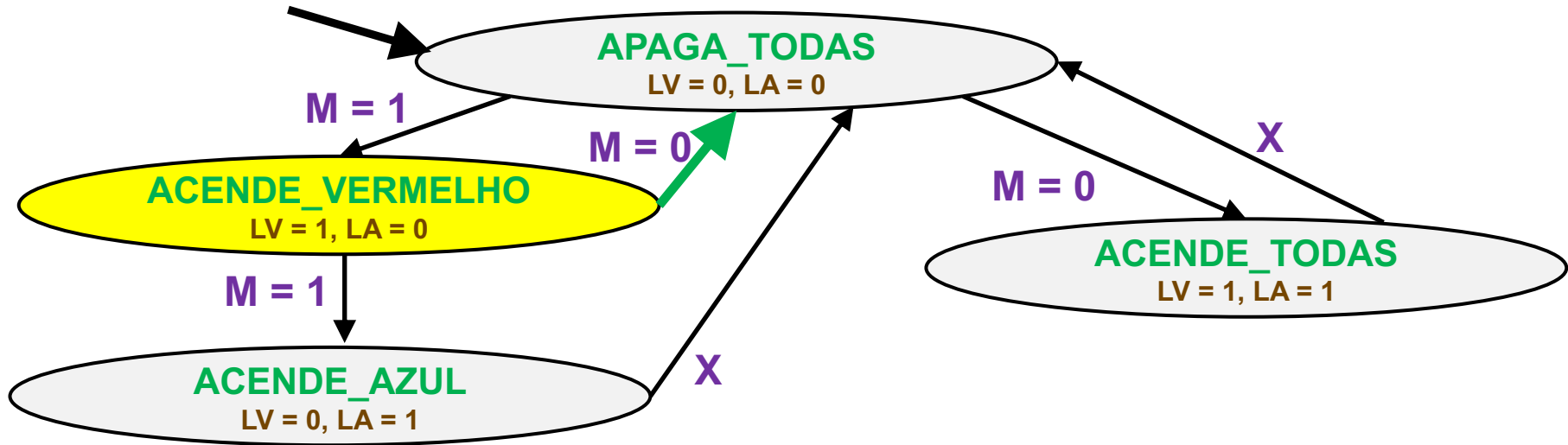
Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0

Obtenção da tabela de estados



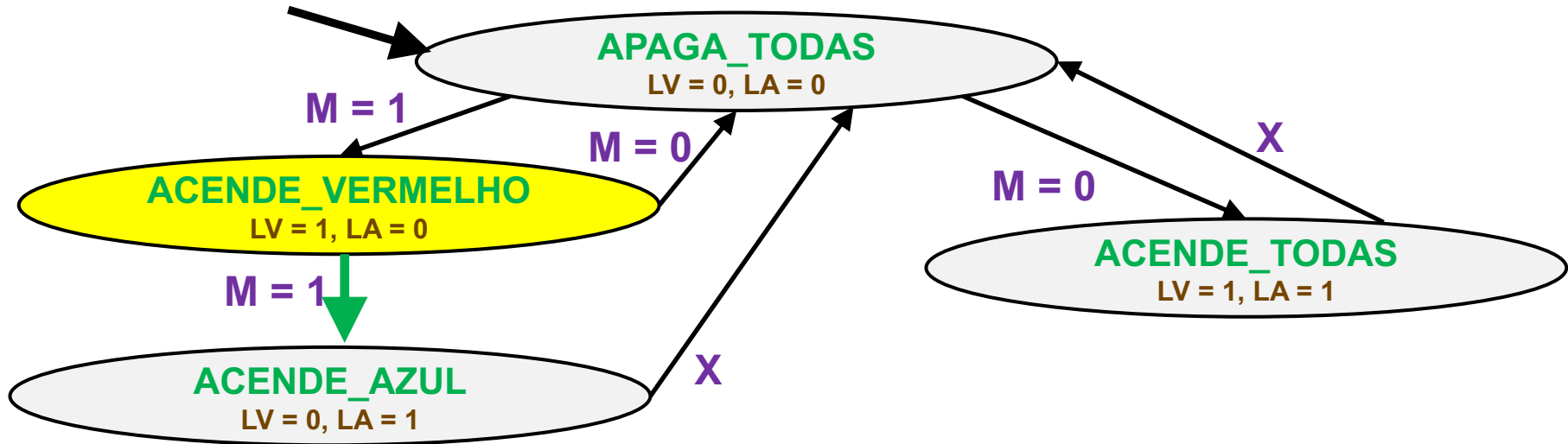
Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0
ACENDE_TODAS	X	APAGA_TODAS	1	1

Obtenção da tabela de estados



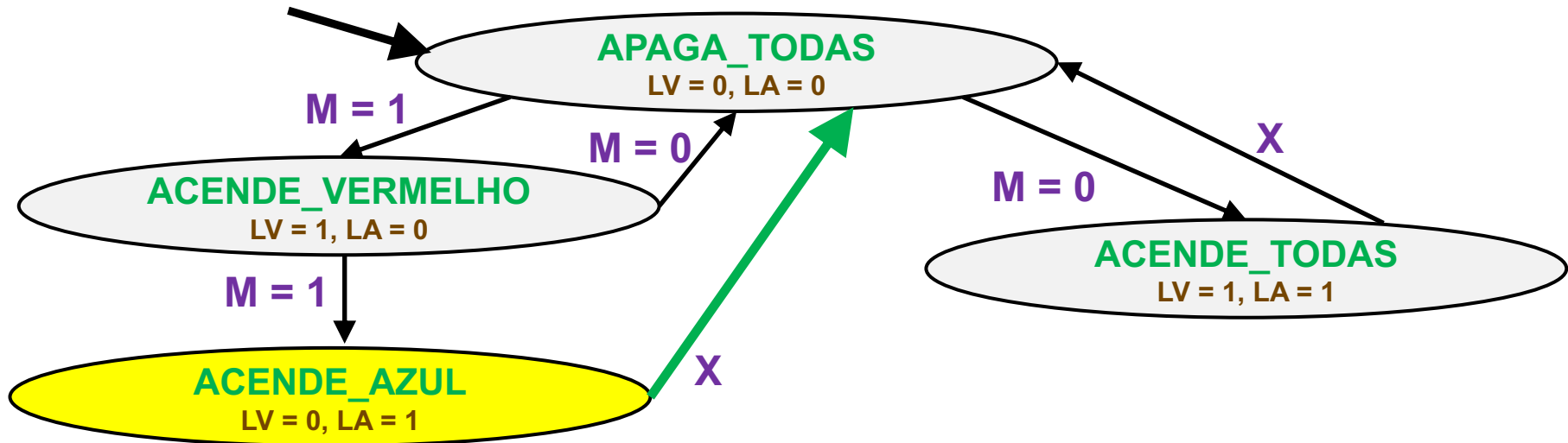
Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0
ACENDE_TODAS	X	APAGA_TODAS	1	1
ACENDE_VERMELHO	0	APAGA_TODAS	1	0

Obtenção da tabela de estados



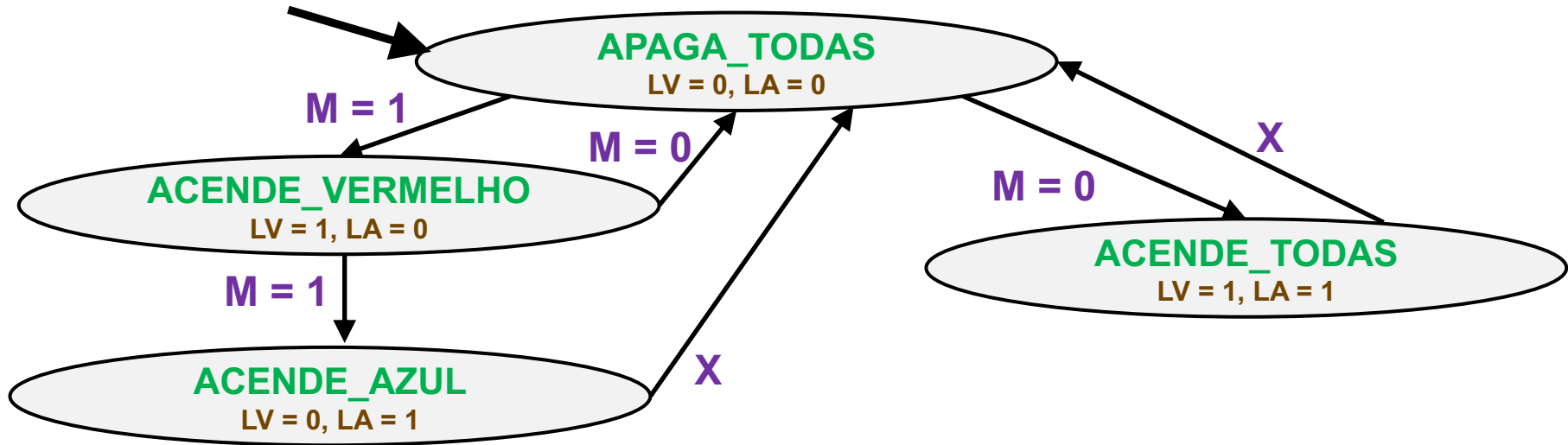
Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0
ACENDE_TODAS	X	APAGA_TODAS	1	1
ACENDE_VERMELHO	0	APAGA_TODAS	1	0
ACENDE_VERMELHO	1	ACENDE_AZUL	1	0

Obtenção da tabela de estados



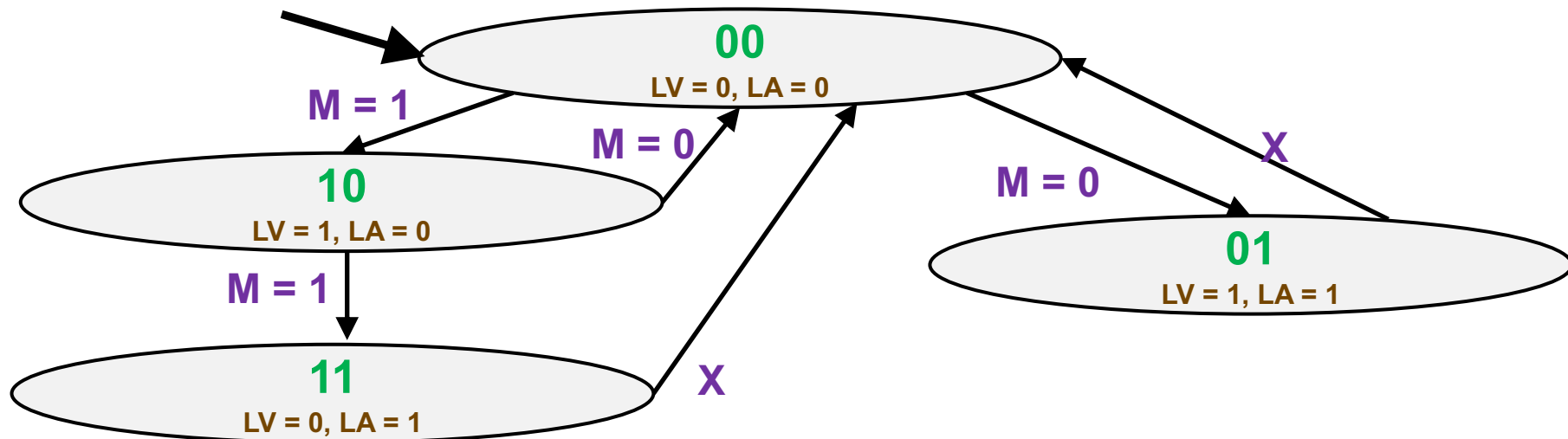
Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0
ACENDE_TODAS	X	APAGA_TODAS	1	1
ACENDE_VERMELHO	0	APAGA_TODAS	1	0
ACENDE_VERMELHO	1	ACENDE_AZUL	1	0
ACENDE_AZUL	X	APAGA_TODAS	0	1

Obtenção da tabela de estados



Estado Atual	Entrada	Próximo Estado	Saída	
			LV	LA
APAGA_TODAS	0	ACENDE_TODAS	0	0
APAGA_TODAS	1	ACENDE_VERMELHO	0	0
ACENDE_TODAS	X	APAGA_TODAS	1	1
ACENDE_VERMELHO	0	APAGA_TODAS	1	0
ACENDE_VERMELHO	1	ACENDE_AZUL	1	0
ACENDE_AZUL	X	APAGA_TODAS	0	1

Codificação dos Estados



Estado Atual		Entrada	Próximo Estado		Saída	
Q_1	Q_0	M	Q_1	Q_0	LV	LA
0	0	0	0	1	0	0
0	0	1	1	0	0	0
0	1	X	0	0	1	1
1	0	0	0	0	1	0
1	0	1	1	1	1	0
1	1	X	0	0	0	1

CÓDIGO DE CONTAGEM BINÁRIA

Estado	Estado do FF (Q_1Q_0)
APAGA_TODAS	00
ACENDE_TODAS	01
ACENDE_VERMELHO	10
ACENDE_AZUL	11

Escolha do Flip-Flop

□ NESTE EXEMPLO VAMOS UTILIZAR O FLIP-FLOP D

ENTRADA(S)

EM UMA MÁQUINA DE MOORE, AS ENTRADAS SÃO USADAS SOMENTE NOS CIRCUITOS COMBINACIONAIS DE PRÓXIMO ESTADO

CLOCK

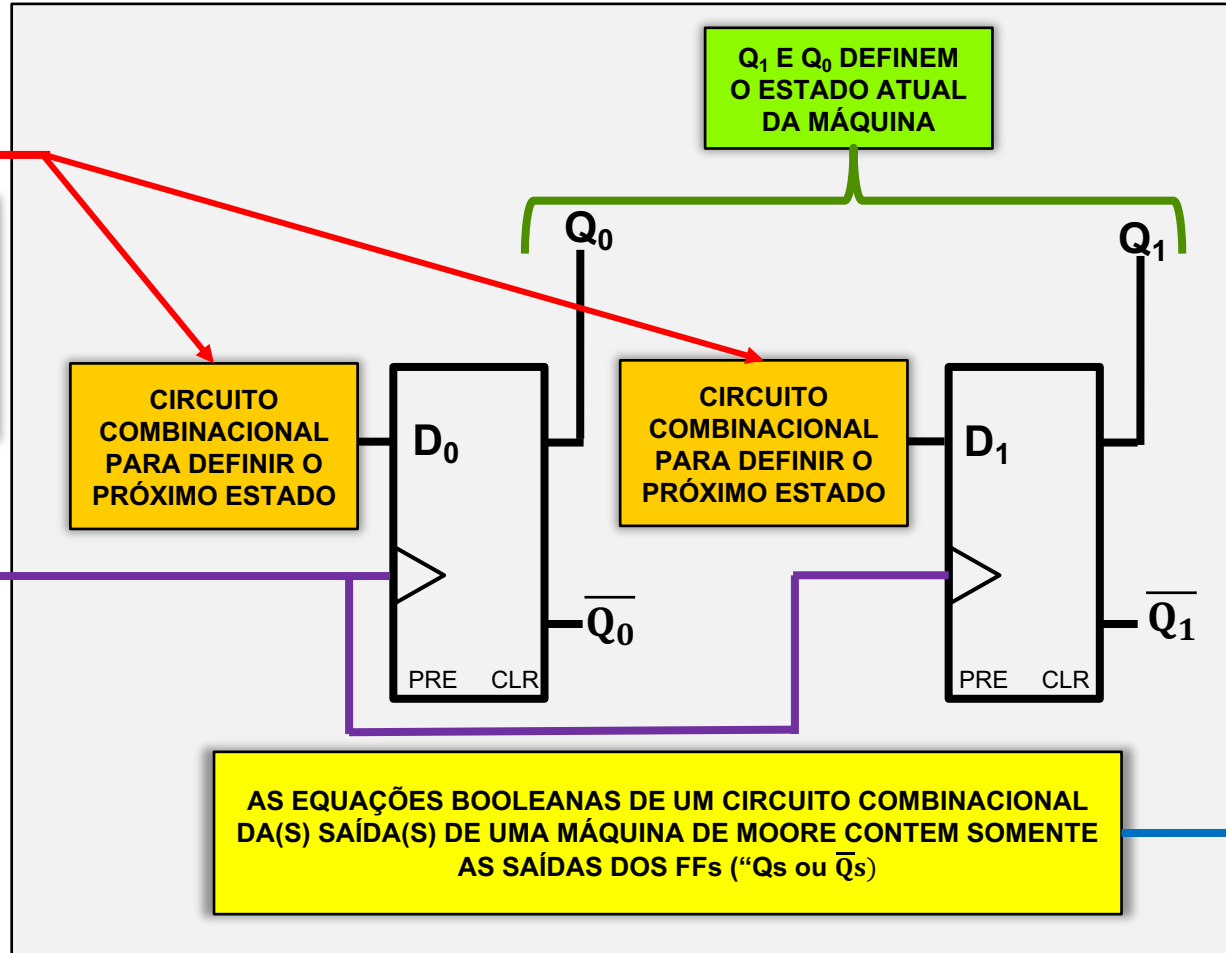


Tabela de Estados

Estado Atual		Entrada	Próximo Estado		Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	Q_1	Q_0	LV	LA	D_1	D_0
0	0	0	0	1	0	0	0	1
0	0	1	1	0	0	0	1	0
0	1	X	0	0	1	1	0	0
1	0	0	0	0	1	0	0	0
1	0	1	1	1	1	0	1	1
1	1	X	0	0	0	1	0	0



Para o FF tipo D, a tabela das equações de entrada é igual a tabela do próximo estado!

Tabela de Estados

Estado Atual		Entrada	Próximo Estado	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M		LV	LA	D_1	D_0
0	0	0	0	0	0	0	1
0	0	1	1	0	0	1	0
0	1	X	0	1	1	0	0
1	0	0	1	1	0	0	0
1	0	1	1	1	0	1	1
1	1	X	0	0	1	0	0

DEPOIS DE PREENCHER A PARTE DA TABELA DE EQUAÇÕES DE ENTRADA, VOCÊ PODE DESCARTAR A PARTE DE PRÓXIMO ESTADO

Tabela de Estados

Estado Atual		Entrada	Saída		Equações de Entrada (FF D)	
Q_1	Q_0	M	LV	LA	D_1	D_0
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	X	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	X	0	1	0	0

Tabela de Estados com ROM

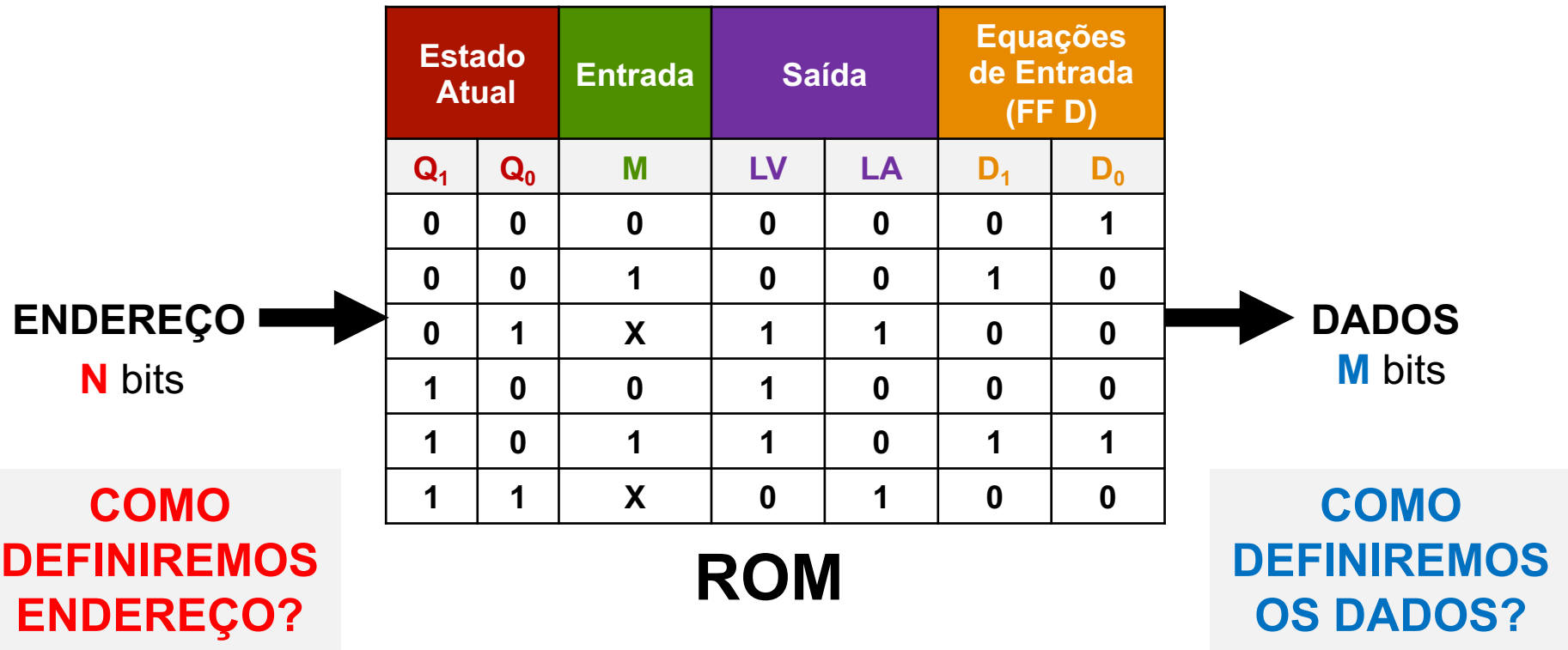


Tabela de Estados com ROM

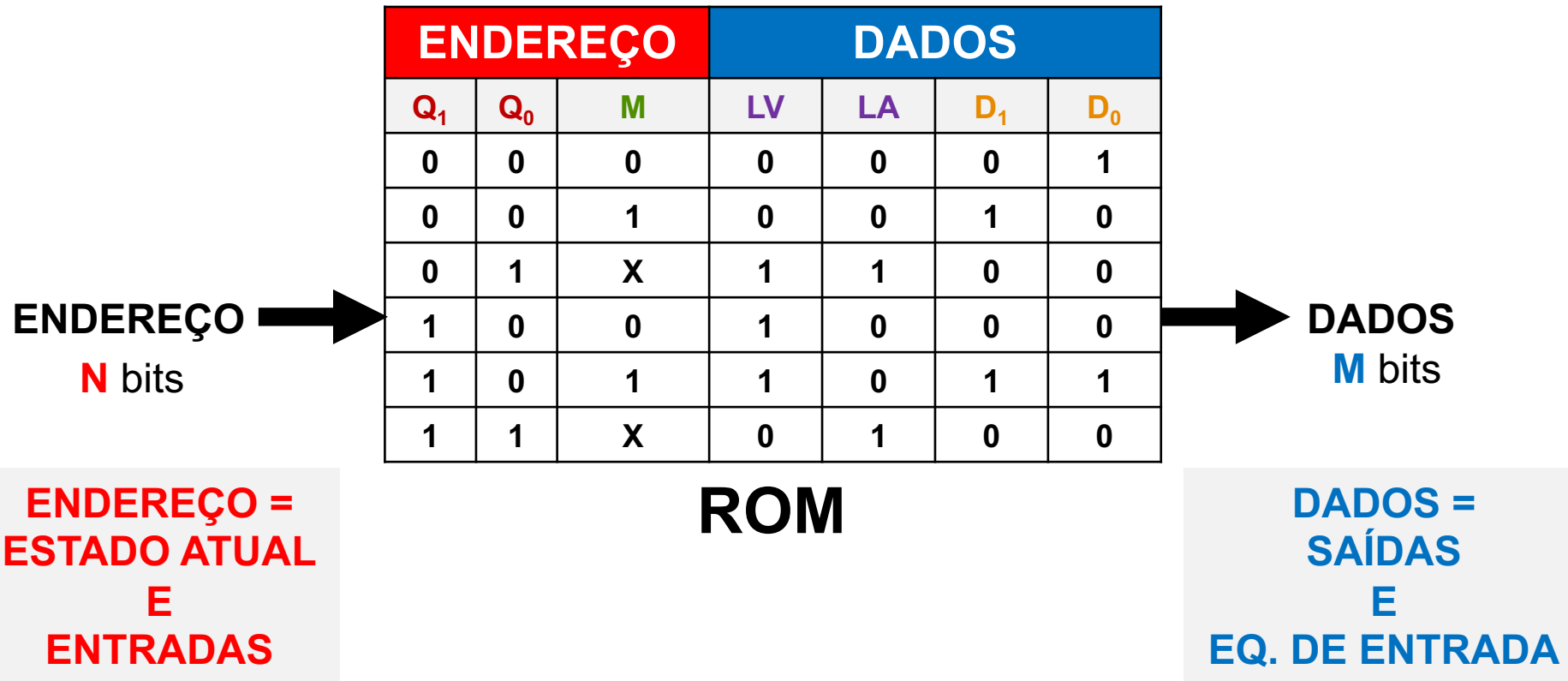
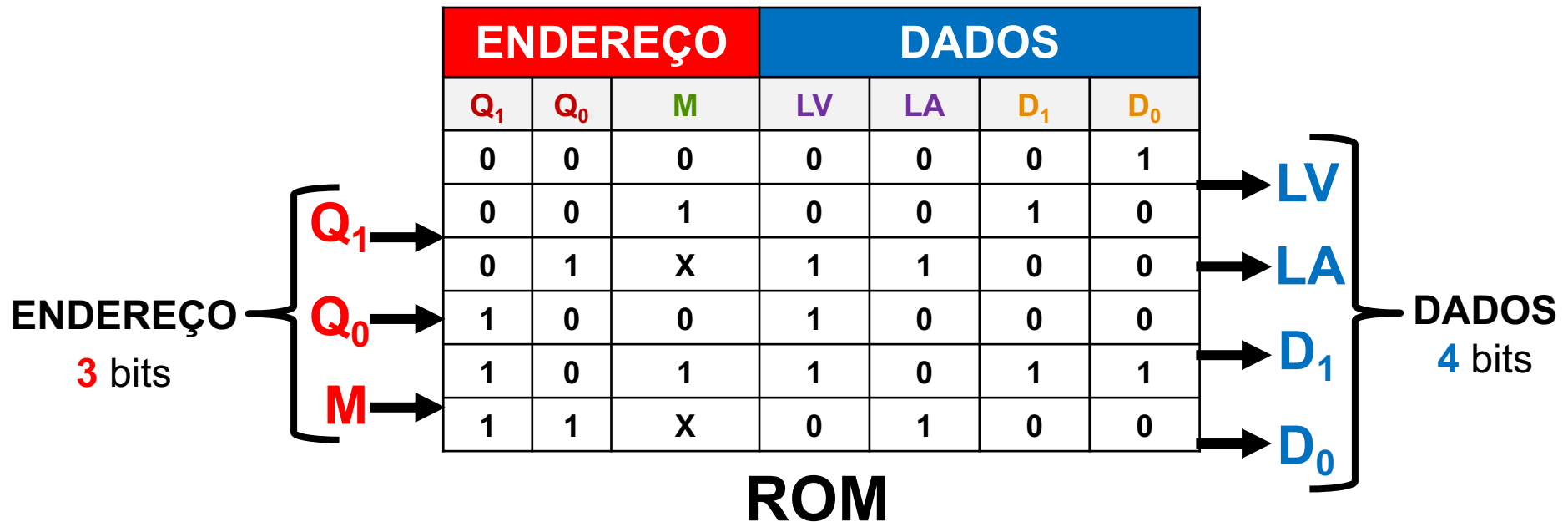
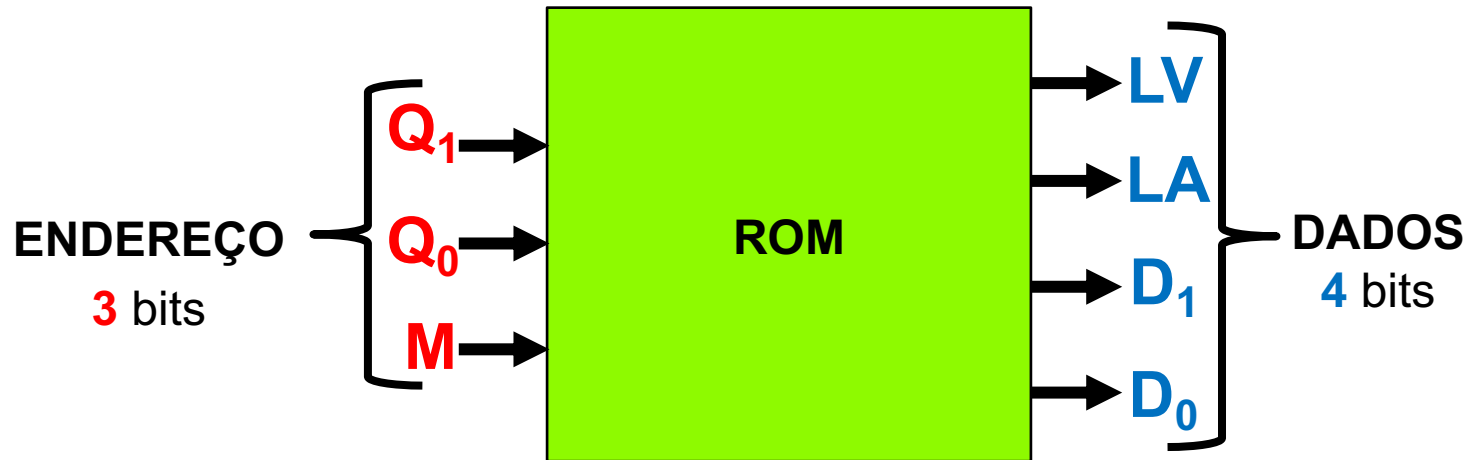


Tabela de Estados com ROM

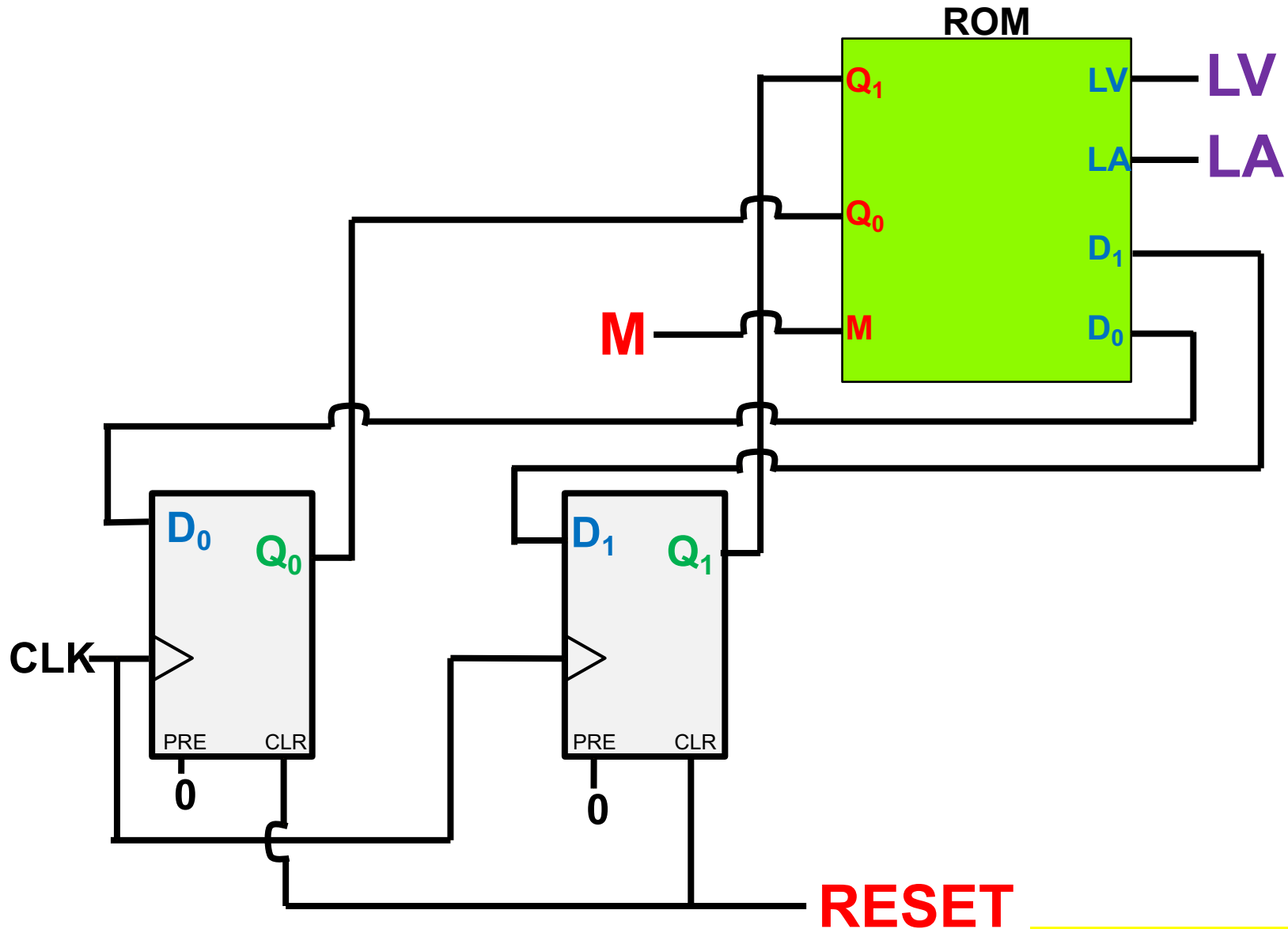


Tamanho da ROM será $4 \times 2^3 = 4 \times 8 = 32$ bits

Tabela de Estados com ROM



Circuito



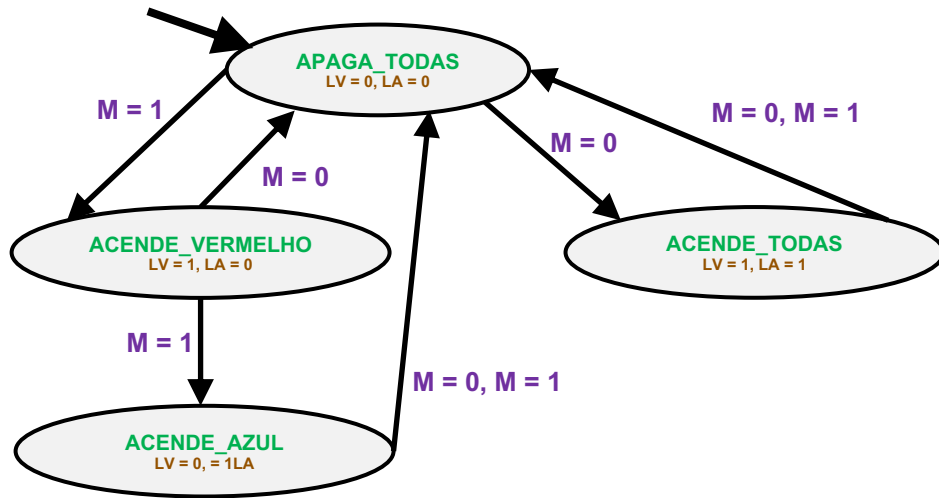
RESET assíncrono!

ROM do Exemplo 3 em SystemVerilog

ENDEREÇO			DADOS			
Q ₁	Q ₀	M	LV	LA	D ₁	D ₀
0	0	0	0	0	0	1
0	0	1	0	0	1	0
0	1	X	1	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	X	0	1	0	0

```
module rom(  
    input [2:0] endereco,  
    output [3:0] dados);  
  
    always_comb  
        casex (endereco)  
            3'b 000: dados = 4'b 0001;  
            3'b 001: dados = 4'b 0010;  
            3'b 01x: dados = 4'b 1100;  
            3'b 100: dados = 4'b 1000;  
            3'b 101: dados = 4'b 1011;  
            3'b 11x: dados = 4'b 0100;  
            default: dados = 4'b 0000;  
        endcase  
  
endmodule
```

Máquinas de Estados em SystemVerilog



```
module fsm(  
    input logic clk,  
    input logic reset,  
    input logic m,  
    output logic la, lv  
);  
  
typedef enum logic [1:0] {APAGA_TODAS, ACENDE_TODAS, ACENDE_VERMELHO, ACENDE_AZUL} statetype;  
statetype state, nextstate;  
  
always_ff @(posedge clk, posedge reset)  
    if (reset) state <= APAGA_TODAS;  
    else state <= nextstate;  
  
always_comb  
    case (state)  
        APAGA_TODAS: if (m) nextstate = ACENDE_VERMELHO;  
                     else nextstate = ACENDE_TODAS;  
        ACENDE_TODAS: nextstate = APAGA_TODAS;  
        ACENDE_VERMELHO: if (m) nextstate = ACENDE_AZUL;  
                          else nextstate = APAGA_TODAS;  
        ACENDE_AZUL: nextstate = APAGA_TODAS;  
        default: nextstate = APAGA_TODAS;  
    endcase  
  
always_comb  
    case (state)  
        APAGA_TODAS: begin  
                        la = 0;  
                        lv = 0;  
                    end  
        ACENDE_TODAS: begin  
                        la = 1;  
                        lv = 1;  
                    end  
        ACENDE_VERMELHO: begin  
                        la = 0;  
                        lv = 1;  
                    end  
        ACENDE_AZUL: begin  
                        la = 1;  
                        lv = 0;  
                    end  
        default: begin  
                        la = 0;  
                        lv = 0;  
                    end  
    endcase  
endmodule
```

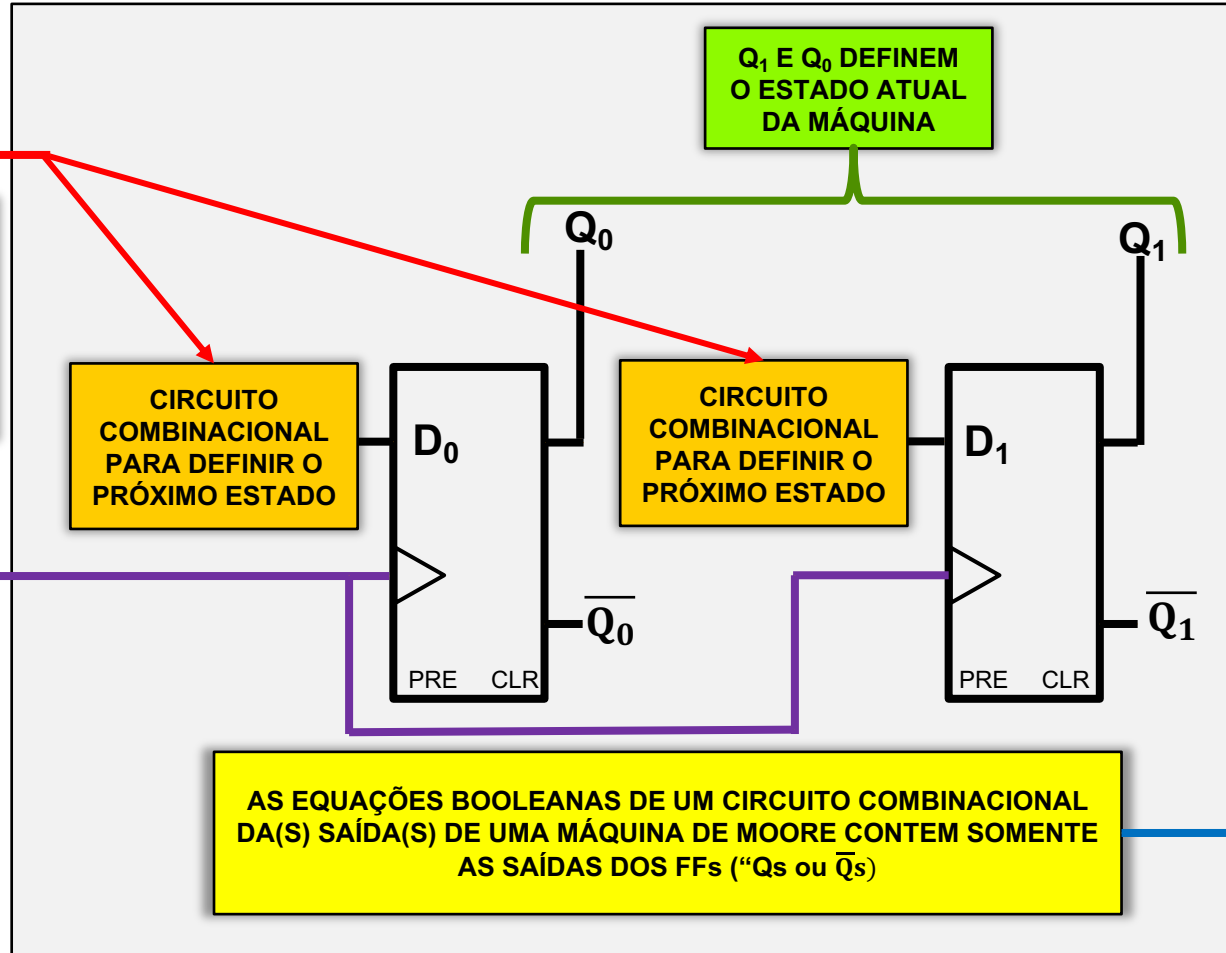
Exemplo Circuito - Máquina de Moore

MÁQUINA DE MOORE (Exemplo com 2 FFs D)

ENTRADA(S)

EM UMA MÁQUINA DE MOORE, AS ENTRADAS SÃO USADAS SOMENTE NOS CIRCUITOS COMBINACIONAIS DE PRÓXIMO ESTADO

CLOCK

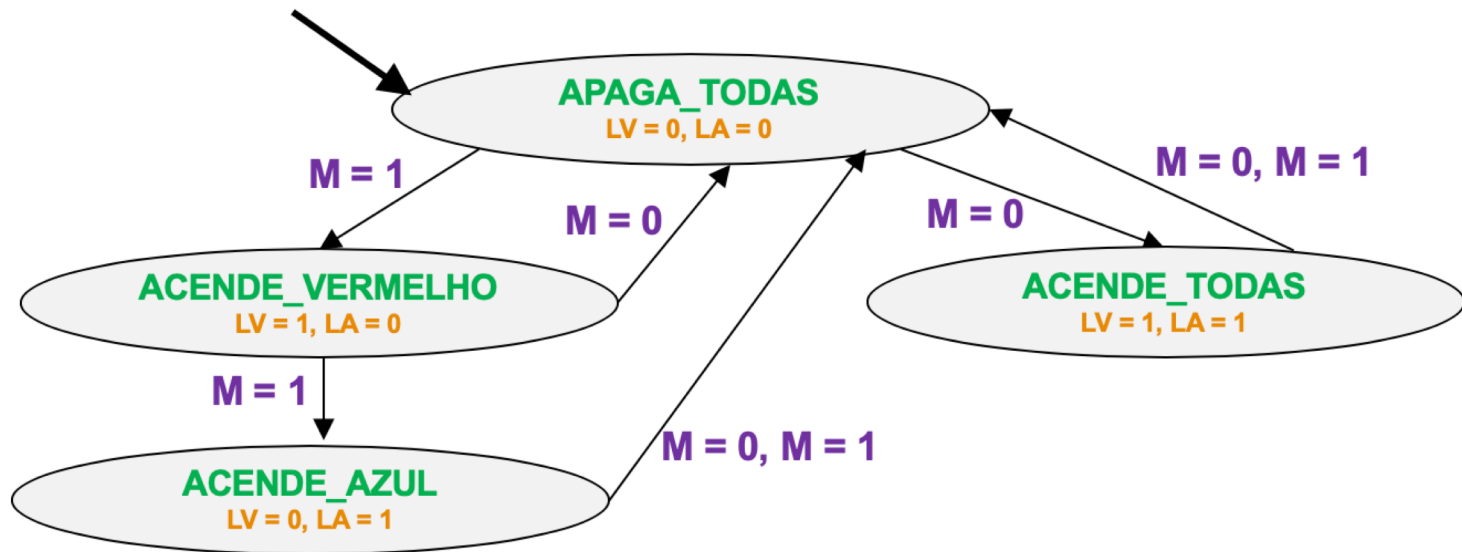


SAÍDA(S)

Máquinas de Estados em SystemVerilog

Definição do módulo

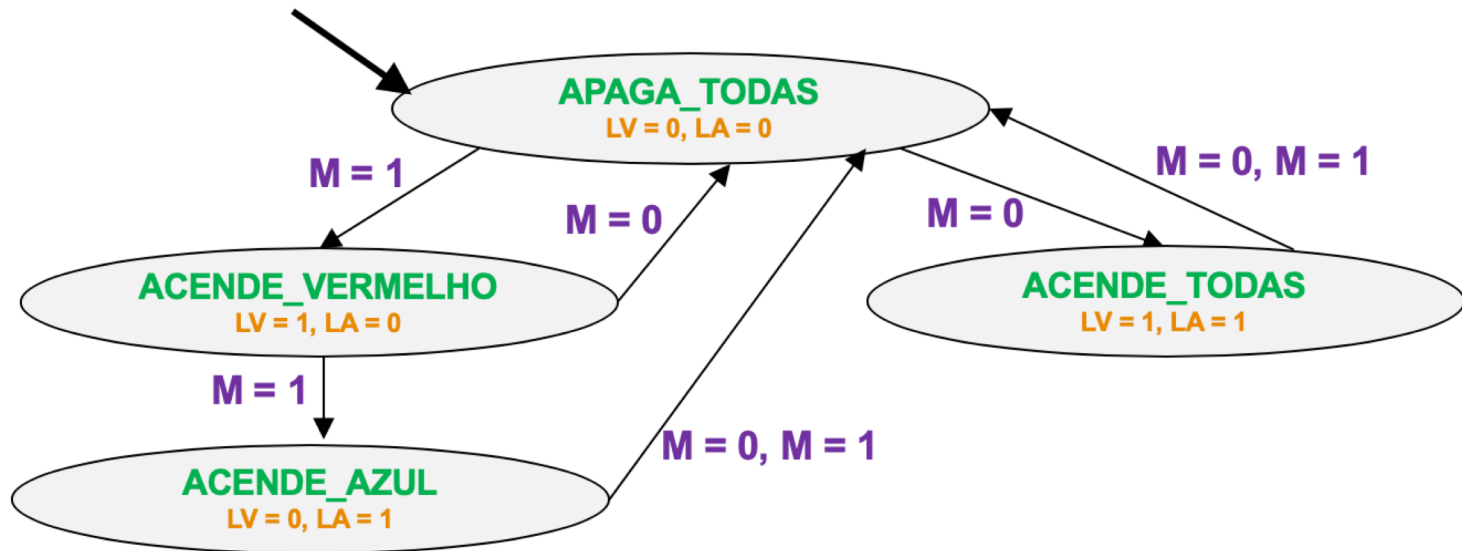
```
module fsm(  
    input logic clk,  
    input logic reset,  
    input logic m,  
    output logic la, lv  
);
```



Máquinas de Estados em SystemVerilog

Definição da variável dos estados

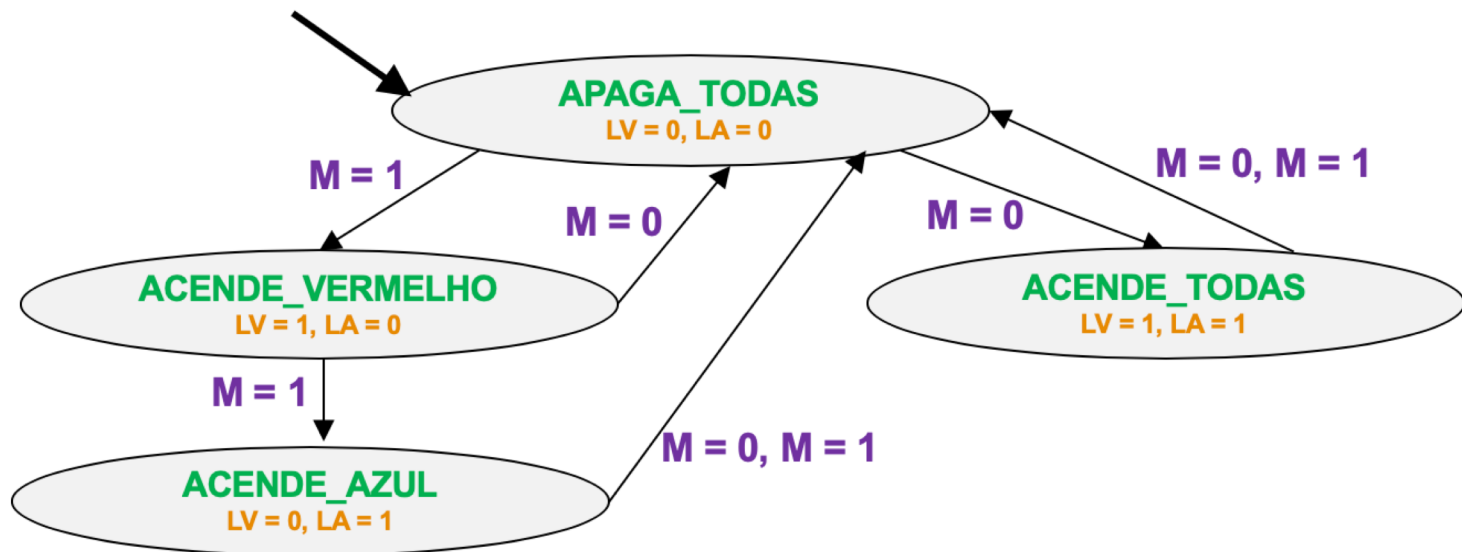
```
typedef enum logic [1:0] {APAGA_TODAS, ACENDE_TODAS, ACENDE_VERMELHO, ACENDE_AZUL} statetype;  
statetype state, nextstate;
```



Máquinas de Estados em SystemVerilog

Definição do registrador de estado

```
always_ff @(posedge clk, posedge reset)
  if (reset) state <= APAGA_TODAS;
  else state <= nextstate;
```



Máquinas de Estados em SystemVerilog

Definição da lógica de próximo estado (Equações de Entrada)

```
always_comb
case (state)

    APAGA_TODAS: if (m) nextstate = ACENDE_VERMELHO;
                  else nextstate = ACENDE_TODAS;

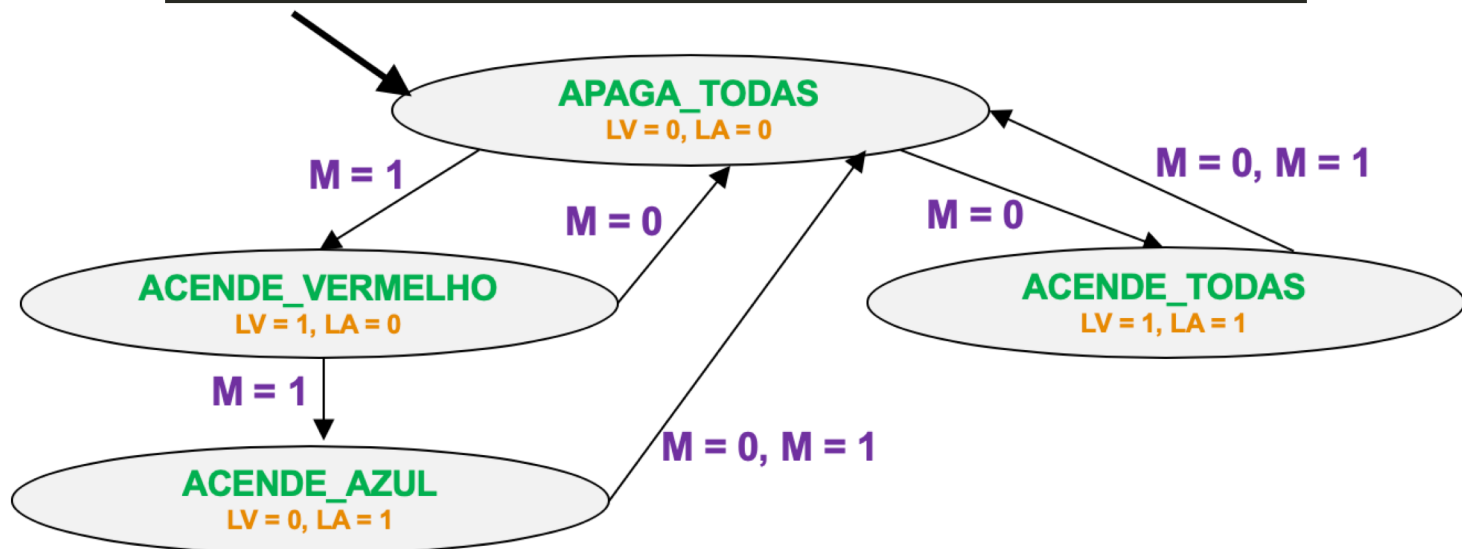
    ACENDE_TODAS: nextstate = APAGA_TODAS;

    ACENDE_VERMELHO: if (m) nextstate = ACENDE_AZUL;
                     else nextstate = APAGA_TODAS;

    ACENDE_AZUL: nextstate = APAGA_TODAS;

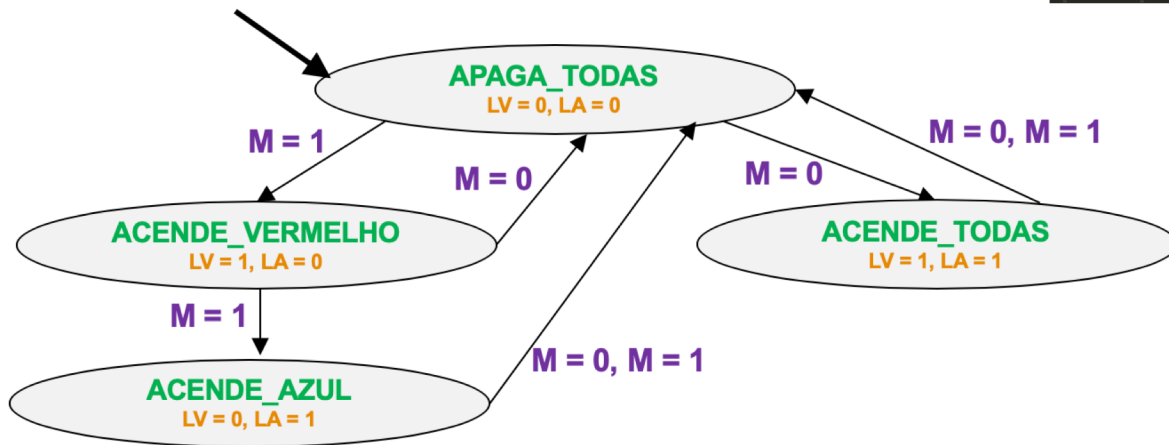
    default: nextstate = APAGA_TODAS;

endcase
```



Máquinas de Estados em SystemVerilog

Definição da lógica de saídas



```
always_comb
case (state)

    APAGA_TODAS:
        begin
            la = 0;
            lv = 0;
        end

    ACENDE_TODAS:
        begin
            la = 1;
            lv = 1;
        end

    ACENDE_VERMELHO:
        begin
            la = 0;
            lv = 1;
        end

    ACENDE_AZUL:
        begin
            la = 1;
            lv = 0;
        end

    default:
        begin
            la = 0;
            lv = 0;
        end

endcase
```

Máquinas de Estados em SystemVerilog

